

Alma Mater Studiorum – Università di Bologna

Corso di Crittografia

Elliptic Curve Cryptography

Mathematical Foundations and Modern
Applications

Studente: **Daniel Alejandro Horna**

Matricola: **0001126664**

Docente: **Prof. Luciano Margara**

Anno accademico: **2025–2026**

Campus di Cesena

Abstract

La crittografia a curve ellittiche rappresenta una delle applicazioni più importanti dell'algebra moderna alla sicurezza informatica. A differenza dei sistemi a chiave pubblica basati sulla fattorizzazione di interi, come RSA, gli schemi basati su curve ellittiche fondano la propria sicurezza sulla difficoltà computazionale del problema del logaritmo discreto su curve ellittiche. Questa caratteristica permette di ottenere livelli di sicurezza elevati con chiavi più corte, rendendo ECC particolarmente adatta a protocolli di rete, dispositivi mobili, sistemi embedded e applicazioni blockchain.

Questa relazione presenta le basi matematiche della crittografia su curve ellittiche, partendo dall'aritmetica modulare, dai gruppi e dai campi finiti, per poi introdurre la definizione di curva ellittica, la legge di gruppo sui punti e la scalar multiplication. Successivamente viene analizzato il problema del logaritmo discreto su curve ellittiche, con particolare attenzione al suo ruolo come assunzione di sicurezza. La parte finale discute alcune applicazioni moderne, tra cui ECDH, ECDSA e Bitcoin, e conclude con una riflessione sui limiti futuri di ECC nel contesto della crittografia post-quantistica.

Indice

Abstract	i
Notazione	vii
1 Introduzione	1
1.1 Motivazione	1
1.2 Perché le curve ellittiche	2
1.3 Obiettivi della relazione	2
1.4 Struttura della relazione	3
2 Fondamenti matematici	4
2.1 Perché servono i fondamenti matematici	4
2.2 Aritmetica modulare	4
2.3 Gruppi	5
2.4 Gruppi abeliani	6
2.5 Gruppi ciclici e generatori	6
2.6 Campi finiti	7
2.7 Inversi modulari	7
2.8 Facile e difficile in senso computazionale	8
2.9 Il problema del logaritmo discreto	8
2.10 Dal logaritmo discreto classico al caso ellittico	9
3 Curve ellittiche sui numeri reali	10
3.1 Introduzione	10
3.2 Definizione generale di curva ellittica	10
3.3 Forma normale di Weierstrass	10
3.4 Condizione di non singolarità	11
3.5 Interpretazione geometrica	12
3.6 Intersezione con le rette	12
3.7 Il punto all'infinito	13
3.8 Perché partire dai numeri reali	13
4 Legge di gruppo sulle curve ellittiche	14
4.1 Introduzione	14
4.2 Idea geometrica della somma	14
4.3 Somma di due punti distinti	15
4.4 Punti opposti e retta verticale	16
4.5 Raddoppio di un punto	16

4.6	Il caso della tangente verticale	17
4.7	Elemento neutro	18
4.8	Inverso di un punto	18
4.9	Proprietà di gruppo	18
4.10	Moltiplicazione scalare	19
4.11	Metodo double-and-add	20
4.12	Importanza crittografica della legge di gruppo	20
5	Curve ellittiche su campi finiti	22
5.1	Introduzione	22
5.2	Curve ellittiche su $\mathbb{F}_p$	22
5.3	Differenza rispetto al caso reale	23
5.4	Esempio di curva su $\mathbb{F}_{17}$	23
5.5	Somma di punti modulo p	23
5.6	Raddoppio di un punto modulo p	24
5.7	Opposti e punto all'infinito	25
5.8	Moltiplicazione scalare e sottogruppi ciclici	25
5.9	Perché i campi finiti sono adatti alla crittografia	26
6	Il problema del logaritmo discreto su curve ellittiche	27
6.1	Introduzione	27
6.2	Richiamo: il logaritmo discreto classico	27
6.3	Dal caso moltiplicativo al caso ellittico	28
6.4	Definizione di ECDLP	29
6.5	Perché la direzione diretta è efficiente	29
6.6	Perché l'inversione è difficile	30
6.7	Esempio su una curva piccola	30
6.8	Ordine del punto e dimensione del sottogruppo	31
6.9	Forza bruta e attacchi generici	31
6.10	ECDLP come funzione one-way	32
6.11	Ruolo di ECDLP nella crittografia a chiave pubblica	32
6.12	Confronto concettuale con RSA	33
6.13	Limite quantistico	33
7	Protocolli crittografici basati su ECC	34
7.1	Introduzione	34
7.2	Parametri pubblici e generazione delle chiavi	34
7.3	Elliptic Curve Diffie–Hellman	35
7.4	Sicurezza e autenticazione in ECDH	36
7.5	ECDHE e forward secrecy	36
7.6	Firme digitali su curve ellittiche	36
7.7	Generazione e verifica di una firma ECDSA	37
7.8	Importanza del nonce in ECDSA	38
7.9	ECDH ed ECDSA a confronto	38

8	Applicazioni moderne di ECC	39
8.1	Introduzione	39
8.2	ECC nei protocolli di comunicazione sicura	39
8.3	Forward secrecy e autenticazione	40
8.4	Crittografia ibrida	40
8.5	ECC in Bitcoin	41
8.6	La curva secp256k1	41
8.7	Wallet, seed phrase e indirizzi	41
8.8	Transazioni, UTXO e firme	42
8.9	ECC e decentralizzazione	42
8.10	Limiti pratici	43
9	Confronto, limiti e prospettiva post-quantistica	44
9.1	Introduzione	44
9.2	ECC e RSA	44
9.3	Efficienza e dimensione delle chiavi	45
9.4	Limiti teorici	45
9.5	Limiti implementativi	45
9.6	Scelta delle curve	46
9.7	La minaccia quantistica	46
9.8	Harvest Now, Decrypt Later	47
9.9	Crittografia post-quantistica	47
9.10	Standardizzazione e transizione	47
9.11	Confronto finale	48
10	Conclusioni	49
10.1	Risultati principali	49
10.2	Rilevanza e limiti	49
10.3	Conclusione finale	50

Elenco delle figure

Elenco delle tabelle

1	Principali simboli utilizzati nella relazione.	vii
2.1	Analogia tra logaritmo discreto classico e logaritmo discreto su curve ellittiche.	9
6.1	Analogia tra logaritmo discreto classico e logaritmo discreto su curve ellittiche.	28
6.2	Problemi computazionali alla base di diversi sistemi a chiave pubblica. .	33
7.1	Confronto tra ECDH ed ECDSA.	38
9.1	Problemi computazionali alla base di alcuni sistemi a chiave pubblica. .	45
9.2	Confronto indicativo tra dimensioni delle chiavi RSA ed ECC.	45
9.3	Confronto qualitativo tra RSA, ECC e crittografia post-quantistica. . .	48

Notazione

Simbolo	Significato
$\mathbb{Z}_p$	Insieme degli interi modulo un primo p
$\mathbb{F}_p$	Campo finito con p elementi
E	Curva ellittica
$E(\mathbb{F}_p)$	Insieme dei punti della curva su $\mathbb{F}_p$
$\mathcal{O}$	Punto all'infinito, elemento neutro del gruppo
$P + Q$	Somma di due punti sulla curva
kP	Moltiplicazione scalare del punto P
$\text{ord}(P)$	Ordine del punto P
ECDLP	Elliptic Curve Discrete Logarithm Problem

Tabella 1: Principali simboli utilizzati nella relazione.

Capitolo 1

Introduzione

1.1 Motivazione

La crittografia moderna nasce dall'esigenza di proteggere informazioni trasmesse o conservate in ambienti nei quali non è realistico assumere la presenza di un canale completamente sicuro. In uno scenario elementare, un mittente vuole comunicare con un destinatario attraverso un mezzo di trasmissione potenzialmente osservabile o manipolabile da un avversario. Il problema crittografico non consiste soltanto nel rendere incomprensibile un messaggio, ma nel garantire proprietà più ampie: confidenzialità, integrità, autenticazione e, in alcuni casi, non ripudio [1, 2].

Nei sistemi a chiave simmetrica, mittente e destinatario utilizzano una stessa informazione segreta per cifrare e decifrare. Questa impostazione è efficiente, ma presenta un limite strutturale: la chiave deve essere condivisa prima della comunicazione attraverso un canale sicuro o mediante un protocollo affidabile di scambio. La crittografia a chiave pubblica nasce per superare questa difficoltà, separando la chiave pubblica, distribuibile liberamente, dalla chiave privata, che deve rimanere segreta [1, 3].

Dal punto di vista matematico, questa separazione richiede funzioni facili da calcolare in una direzione ma difficili da invertire senza un'informazione segreta. La sicurezza non deriva dalla segretezza dell'algoritmo, ma dalla difficoltà computazionale di risolvere determinati problemi matematici. In questo senso, la crittografia moderna offre sicurezza computazionale: un attacco può esistere in teoria, ma richiedere risorse tali da non essere praticabile [1, 4, 3].

Questa idea è centrale per comprendere la crittografia su curve ellittiche. ECC non è un singolo cifrario, ma una famiglia di costruzioni basate sulla struttura algebrica dei punti di una curva ellittica definita su un campo finito. Il principio fondamentale è il seguente: dato un punto P sulla curva e uno scalare segreto k , è efficiente calcolare

$$Q = kP.$$

Al contrario, dato soltanto P e Q , risulta computazionalmente difficile recuperare k . Questa asimmetria costituisce il nucleo del problema del logaritmo discreto su curve ellittiche [5, 6].

1.2 Perché le curve ellittiche

Le curve ellittiche sono oggetti matematici definiti mediante equazioni algebriche. Nella forma di Weierstrass, una curva ellittica su un campo di caratteristica diversa da 2 e da 3 può essere espressa come

$$y^2 = x^3 + ax + b. \quad (1.1)$$

La proprietà decisiva è che i punti della curva possono essere dotati di una legge di composizione che li trasforma in un gruppo abeliano. Questa legge permette di sommare punti e, ripetendo la somma di uno stesso punto, di definire la moltiplicazione scalare kP [5].

Nel passaggio dalla geometria alla crittografia, il ruolo centrale è assunto dai campi finiti. Le curve ellittiche usate nei sistemi crittografici non sono curve continue disegnate sul piano reale, ma insiemi finiti di punti con coordinate in un campo come $\mathbb{F}_p$. Questo consente di evitare problemi di approssimazione numerica e di lavorare con operazioni esatte, efficienti e implementabili in software e hardware [5, 6].

L'interesse pratico di ECC deriva dal fatto che, rispetto ad altri sistemi a chiave pubblica come RSA, permette di ottenere livelli di sicurezza comparabili con chiavi significativamente più brevi. Questo riduce memoria, larghezza di banda e costo computazionale, rendendo ECC adatta a protocolli di rete, dispositivi mobili, smart card, sistemi embedded e applicazioni distribuite [7, 6].

L'importanza di ECC è evidente anche in sistemi reali. In Bitcoin, ad esempio, le chiavi private e pubbliche sono costruite mediante crittografia asimmetrica su curve ellittiche, e la curva utilizzata è secp256k1, definita dall'equazione

$$y^2 = x^3 + 7 \quad (1.2)$$

su un grande campo primo. Questo mostra come una costruzione matematica astratta possa diventare parte essenziale di un sistema economico decentralizzato [8, 9].

1.3 Obiettivi della relazione

L'obiettivo di questa relazione è presentare la crittografia su curve ellittiche in modo progressivo, collegando le basi matematiche alle applicazioni crittografiche. La relazione non si limita a descrivere l'uso pratico di ECC, ma cerca di chiarire perché questa famiglia di protocolli sia sicura, quali assunzioni matematiche utilizzi e quali siano i suoi limiti.

In particolare, il lavoro si propone di:

1. introdurre i concetti algebrici necessari, tra cui aritmetica modulare, gruppi, campi finiti e logaritmo discreto;
2. definire le curve ellittiche sui numeri reali e sui campi finiti;
3. spiegare la legge di gruppo sui punti della curva;
4. analizzare la moltiplicazione scalare e il problema del logaritmo discreto su curve ellittiche;

5. descrivere protocolli basati su ECC, in particolare ECDH ed ECDSA;
6. discutere applicazioni reali, con attenzione a Bitcoin;
7. valutare i limiti di ECC, inclusa la vulnerabilità teorica rispetto agli algoritmi quantistici.

La relazione adotta quindi una prospettiva duplice: matematica, perché la sicurezza dipende dalla struttura algebrica; applicativa, perché tali strutture sono oggi integrate in protocolli e sistemi concreti.

1.4 Struttura della relazione

La relazione è organizzata in tre parti principali. La prima parte introduce i fondamenti matematici: aritmetica modulare, gruppi, campi finiti, curve ellittiche sui reali e legge di gruppo. La seconda parte trasferisce questi concetti ai campi finiti e analizza il problema del logaritmo discreto su curve ellittiche, che costituisce la base della sicurezza di ECC. La terza parte descrive protocolli e applicazioni, in particolare ECDH, ECDSA e Bitcoin, per poi concludere con un confronto con RSA e una discussione sulla prospettiva post-quantistica.

Capitolo 2

Fondamenti matematici

2.1 Perché servono i fondamenti matematici

La crittografia su curve ellittiche utilizza concetti matematici che, a prima vista, possono sembrare lontani dall'informatica applicata: aritmetica modulare, gruppi, campi finiti e logaritmi discreti. Tuttavia, questi strumenti non sono elementi accessori. Essi permettono di costruire operazioni che sono efficienti da calcolare in una direzione, ma difficili da invertire senza conoscere un'informazione segreta.

Questa asimmetria computazionale è alla base della crittografia a chiave pubblica [4, 3]. In un protocollo crittografico moderno, non basta che una funzione sia complicata: deve essere facile da usare per gli utenti legittimi e impraticabile da invertire per un avversario. Per questo motivo, prima di introdurre formalmente le curve ellittiche, è necessario richiamare alcune strutture algebriche fondamentali.

Lo scopo di questo capitolo non è fornire una trattazione completa di algebra astratta, ma introdurre gli strumenti necessari per comprendere i capitoli successivi. In particolare, verranno presentati l'aritmetica modulare, la nozione di gruppo, i gruppi abeliani e ciclici, i campi finiti, gli inversi modulari e il problema del logaritmo discreto.

2.2 Aritmetica modulare

L'aritmetica modulare studia gli interi considerando equivalenti due numeri che hanno lo stesso resto nella divisione per un intero positivo fissato. Se n è un intero positivo, si scrive

$$a \equiv b \pmod{n} \tag{2.1}$$

quando a e b hanno lo stesso resto nella divisione per n . Equivalentemente, $a \equiv b \pmod{n}$ se e solo se n divide $a - b$ [3].

Esempio 2.1. *Si ha*

$$20 \equiv 6 \pmod{7},$$

perché $20 - 6 = 14$ è divisibile per 7. Infatti, sia 20 sia 6 hanno resto 6 nella divisione per 7.

Modulo n , ogni intero è equivalente a uno degli elementi

$$\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}.$$

Per esempio, modulo 7, tutti gli interi possono essere rappresentati mediante l'insieme

$$\mathbb{Z}_7 = \{0, 1, 2, 3, 4, 5, 6\}.$$

Questa riduzione è importante perché permette di lavorare con insiemi finiti. In crittografia, gli algoritmi devono operare su dati rappresentabili in memoria e manipolabili in tempo finito; l'aritmetica modulare consente di mantenere i risultati all'interno di un dominio controllato.

Le operazioni di somma e prodotto modulo n si comportano in modo compatibile con le operazioni ordinarie [3]:

$$(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n, \quad (2.2)$$

$$(ab) \bmod n = ((a \bmod n)(b \bmod n)) \bmod n. \quad (2.3)$$

Questo permette di ridurre continuamente i risultati intermedi senza modificare il risultato finale modulo n .

2.3 Gruppi

Un gruppo è una struttura algebrica composta da un insieme non vuoto e da un'operazione interna. L'idea fondamentale è che l'operazione rimanga sempre dentro l'insieme e che sia possibile definire un elemento neutro e un inverso per ogni elemento [10, 5].

Definizione 2.1 (Gruppo). *Una coppia $(G, *)$ è un gruppo se valgono le seguenti proprietà:*

1. **Chiusura:** per ogni $a, b \in G$, anche $a * b \in G$;

2. **Associatività:** per ogni $a, b, c \in G$, vale

$$(a * b) * c = a * (b * c);$$

3. **Elemento neutro:** esiste $e \in G$ tale che

$$a * e = e * a = a$$

per ogni $a \in G$;

4. **Inverso:** per ogni $a \in G$, esiste $a^{-1} \in G$ tale che

$$a * a^{-1} = a^{-1} * a = e.$$

Esempio 2.2. *L'insieme $\mathbb{Z}_n$ con l'operazione di somma modulo n è un gruppo. L'elemento neutro è 0, e l'inverso additivo di un elemento a è l'elemento che sommato ad a dà 0 modulo n .*

Per esempio, in $\mathbb{Z}_7$, l'inverso additivo di 3 è 4, perché

$$3 + 4 \equiv 0 \pmod{7}.$$

Nei capitoli successivi non useremo soltanto gruppi numerici. Il punto essenziale è che anche l'insieme dei punti di una curva ellittica, con una particolare operazione di somma, può essere organizzato come un gruppo.

2.4 Gruppi abeliani

Un gruppo è detto abeliano se l'operazione è commutativa. Questo significa che per ogni $a, b \in G$ vale

$$a * b = b * a. \quad (2.4)$$

La commutatività non è richiesta nella definizione generale di gruppo, ma sarà importante nel nostro caso: i punti di una curva ellittica, con l'operazione di somma, formano un gruppo abeliano [5].

Esempio 2.3. *Il gruppo $(\mathbb{Z}_n, +)$ è abeliano, perché*

$$a + b \equiv b + a \pmod{n}.$$

L'aggettivo “abeliano” indica quindi che l'ordine con cui vengono combinati due elementi non cambia il risultato. Questa proprietà renderà più naturale interpretare la somma di punti su una curva ellittica.

2.5 Gruppi ciclici e generatori

Un gruppo G è ciclico se esiste un elemento $g \in G$ tale che tutti gli elementi del gruppo possono essere ottenuti applicando ripetutamente l'operazione del gruppo a g . In notazione moltiplicativa, si scrive

$$G = \langle g \rangle = \{g^i : i \in \mathbb{Z}\}. \quad (2.5)$$

L'elemento g prende il nome di generatore.

Nel caso additivo, che sarà quello usato per le curve ellittiche, la stessa idea viene scritta come

$$\langle P \rangle = \{P, 2P, 3P, \dots\}, \quad (2.6)$$

dove

$$2P = P + P, \quad 3P = P + P + P,$$

e così via. In questo contesto, P è un generatore del sottogruppo formato dai suoi multipli.

In ECC si lavora spesso in un sottogruppo ciclico generato da un punto base pubblico [10, 5].

2.6 Campi finiti

Un campo è una struttura algebrica nella quale sono definite due operazioni, somma e prodotto, con proprietà analoghe a quelle note per i numeri razionali o reali. In un campo è possibile sommare, sottrarre, moltiplicare e dividere per ogni elemento non nullo [5].

Definizione 2.2 (Campo). *Un insieme K , dotato di due operazioni $+$ e $\cdot$, è un campo se:*

1. $(K, +)$ è un gruppo abeliano con elemento neutro 0;
2. $(K \setminus \{0\}, \cdot)$ è un gruppo abeliano con elemento neutro 1;
3. vale la proprietà distributiva:

$$a(b + c) = ab + ac$$

per ogni $a, b, c \in K$.

Un campo è finito se contiene un numero finito di elementi. Se p è un numero primo, l'insieme

$$\mathbb{Z}_p = \{0, 1, 2, \dots, p-1\} \quad (2.7)$$

con somma e prodotto modulo p è un campo finito [5]. In letteratura, lo stesso campo viene spesso indicato anche con $\mathbb{F}_p$. In questa relazione useremo entrambe le notazioni: $\mathbb{Z}_p$ per sottolineare la rappresentazione tramite classi di resto modulo p , e $\mathbb{F}_p$ quando vorremo evidenziare la struttura di campo.

La primalità di p è essenziale. Se p è primo, ogni elemento non nullo di $\mathbb{Z}_p$ possiede un inverso moltiplicativo. Se invece il modulo non è primo, possono esistere elementi non nulli privi di inverso.

Esempio 2.4. In $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$, l'elemento 2 ha inverso 3, perché

$$2 \cdot 3 = 6 \equiv 1 \pmod{5}.$$

Tutti gli elementi non nulli di $\mathbb{Z}_5$ hanno un inverso moltiplicativo.

Le curve ellittiche usate in crittografia sono definite su campi finiti. In questa relazione ci concentreremo soprattutto sui campi primi $\mathbb{F}_p$, sufficienti per comprendere molte costruzioni moderne, inclusa la curva secp256k1 usata in Bitcoin [5, 8].

2.7 Inversi modulari

L'inverso modulare è lo strumento che permette di effettuare divisioni in aritmetica modulare. Dato un intero a e un modulo primo p , l'inverso di a modulo p è un elemento a^{-1} tale che

$$aa^{-1} \equiv 1 \pmod{p}. \quad (2.8)$$

Quando p è primo e $a \not\equiv 0 \pmod{p}$, tale inverso esiste sempre [3, 5].

Esempio 2.5. In $\mathbb{Z}_{17}$, l'inverso di 5 è 7, perché

$$5 \cdot 7 = 35 \equiv 1 \pmod{17}.$$

Quindi dividere per 5 modulo 17 equivale a moltiplicare per 7 modulo 17.

Questo concetto sarà usato direttamente nella somma di punti su curve ellittiche. Per esempio, la formula del coefficiente angolare tra due punti contiene una divisione:

$$\lambda = \frac{y_Q - y_P}{x_Q - x_P}. \quad (2.9)$$

Su un campo finito, questa divisione viene interpretata come moltiplicazione per l'inverso modulare:

$$\lambda \equiv (y_Q - y_P)(x_Q - x_P)^{-1} \pmod{p}. \quad (2.10)$$

2.8 Facile e difficile in senso computazionale

In crittografia, dire che un problema è facile o difficile non significa riferirsi all'intuizione umana. Il significato è computazionale. Un problema è considerato facile se esiste un algoritmo efficiente per risolverlo, tipicamente in tempo polinomiale rispetto alla dimensione dell'input. È considerato difficile se i migliori algoritmi noti richiedono risorse troppo elevate per essere praticabili su input di dimensione crittografica [4].

Questa distinzione è alla base della crittografia a chiave pubblica. Si cercano operazioni che siano facili nella direzione legittima, ma difficili nella direzione dell'attacco. Per esempio, moltiplicare due grandi numeri primi è facile, mentre fattorizzare il loro prodotto è ritenuto difficile. Analogamente, calcolare una potenza modulare è facile, mentre risolvere il logaritmo discreto può essere difficile in gruppi opportunamente scelti [4, 3].

La nozione di difficoltà non implica che l'attacco sia logicamente impossibile. Significa piuttosto che il costo dell'attacco supera le risorse realisticamente disponibili. Per questo motivo, la sicurezza crittografica è normalmente una sicurezza pratica, fondata sul costo computazionale.

2.9 Il problema del logaritmo discreto

Consideriamo un numero primo p e il gruppo moltiplicativo degli elementi non nulli modulo p :

$$\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}. \quad (2.11)$$

Dato un elemento $g \in \mathbb{Z}_p^*$ e un intero x , è efficiente calcolare

$$y = g^x \pmod{p}. \quad (2.12)$$

Il problema inverso consiste nel determinare x conoscendo g , y e p . Questo problema prende il nome di logaritmo discreto [4, 10].

Definizione 2.3 (Logaritmo discreto). *Dato un gruppo finito G , un elemento $g \in G$ e un elemento $y \in \langle g \rangle$, il logaritmo discreto di y in base g è un intero x tale che*

$$y = g^x.$$

Nel caso modulare, l'equazione diventa

$$y \equiv g^x \pmod{p}.$$

Esempio 2.6. *Sia $p = 17$. Nel gruppo $\mathbb{Z}_{17}^*$, si ha*

$$3^4 = 81 \equiv 13 \pmod{17}.$$

La direzione facile è calcolare 13 a partire da 3 e 4. La direzione inversa è: dati 3 e 13, trovare l'esponente 4. Su numeri piccoli si può procedere per tentativi, ma su parametri crittografici il problema diventa computazionalmente difficile.

Il problema del logaritmo discreto è centrale in diversi protocolli crittografici, tra cui Diffie–Hellman ed ElGamal [10, 11]. La crittografia su curve ellittiche ne utilizza una variante più sofisticata, definita non su un gruppo moltiplicativo modulo p , ma sul gruppo dei punti di una curva ellittica.

2.10 Dal logaritmo discreto classico al caso ellittico

ECC utilizza un'idea analoga al logaritmo discreto classico, ma sostituisce il gruppo moltiplicativo modulo p con il gruppo additivo dei punti di una curva ellittica su un campo finito [5].

Nel caso classico, l'operazione facile è

$$y = g^x \bmod p. \quad (2.13)$$

Nel caso ellittico, l'operazione corrispondente è

$$Q = kP, \quad (2.14)$$

dove P è un punto della curva, k è uno scalare intero e kP significa sommare P con se stesso k volte.

L'analogia può essere riassunta nella seguente tabella:

Contesto	Operazione facile	Problema inverso difficile
Logaritmo discreto classico	$g^x \bmod p$	trovare x da g e g^x
Curve ellittiche	kP	trovare k da P e kP

Tabella 2.1: Analogia tra logaritmo discreto classico e logaritmo discreto su curve ellittiche.

Il secondo problema è chiamato *Elliptic Curve Discrete Logarithm Problem*, abbreviato in ECDLP. Esso sarà analizzato in dettaglio nel Capitolo 6, ma è utile introdurlo già qui perché spiega perché gruppi, campi finiti e aritmetica modulare sono necessari per costruire ECC [5, 6].

Capitolo 3

Curve ellittiche sui numeri reali

3.1 Introduzione

Prima di studiare le curve ellittiche usate in crittografia, è utile considerarle sui numeri reali. Questa scelta non corrisponde all'ambiente effettivamente utilizzato nei protocolli crittografici, ma permette di visualizzare geometricamente la struttura della curva e di comprendere in modo intuitivo l'operazione di somma tra punti.

In crittografia, infatti, le curve ellittiche vengono definite su campi finiti, dove non esiste una rappresentazione continua nel piano cartesiano. Tuttavia, molte idee fondamentali, come il punto all'infinito, la simmetria rispetto all'asse delle ascisse e la regola geometrica “retta e riflessione”, sono più semplici da introdurre sui reali. Una volta compresa questa intuizione geometrica, sarà più naturale passare alla formulazione algebrica su campi finiti [5].

3.2 Definizione generale di curva ellittica

Una curva ellittica è un insieme di punti che soddisfano una particolare equazione algebrica. Nella forma più generale, una curva ellittica E definita su un campo K può essere descritta da un'equazione cubica del tipo

$$y^2 + axy + by = x^3 + cx^2 + dx + e, \quad (3.1)$$

dove $a, b, c, d, e \in K$ [5].

Questa forma generale è utile dal punto di vista teorico, ma nella maggior parte delle applicazioni crittografiche e nella trattazione introduttiva si utilizza una forma più semplice, detta forma normale di Weierstrass. Tale forma può essere ottenuta, sotto opportune condizioni sul campo, mediante cambiamenti di variabile.

3.3 Forma normale di Weierstrass

Se il campo su cui è definita la curva ha caratteristica diversa da 2 e da 3, l'equazione di una curva ellittica può essere scritta nella forma normale di Weierstrass [5, 6]:

$$y^2 = x^3 + ax + b, \quad (3.2)$$

dove $a, b \in K$.

Nel caso dei numeri reali, cioè $K = \mathbb{R}$, la curva è l'insieme dei punti $(x, y) \in \mathbb{R}^2$ che soddisfano l'equazione

$$E(\mathbb{R}) = \{(x, y) \in \mathbb{R}^2 : y^2 = x^3 + ax + b\}. \quad (3.3)$$

A questo insieme viene aggiunto un punto speciale, chiamato punto all'infinito e indicato con $\mathcal{O}$ [5]. La curva ellittica completa sui reali viene quindi considerata come

$$E(\mathbb{R}) = \{(x, y) \in \mathbb{R}^2 : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}. \quad (3.4)$$

Il punto $\mathcal{O}$ non è un punto ordinario del piano cartesiano. Esso viene introdotto per rendere ben definita la struttura di gruppo che sarà studiata nel capitolo successivo.

3.4 Condizione di non singolarità

Non ogni equazione della forma $y^2 = x^3 + ax + b$ definisce una curva ellittica valida. È necessario escludere i casi in cui la curva presenta punti singolari, come cuspidi o nodi. In tali punti, la tangente non è definita in modo univoco e la legge di somma tra punti diventerebbe problematica.

Per evitare questi casi, si richiede la condizione di non singolarità:

$$4a^3 + 27b^2 \neq 0. \quad (3.5)$$

Questa condizione garantisce che il polinomio cubico

$$f(x) = x^3 + ax + b \quad (3.6)$$

non abbia radici multiple. Dal punto di vista geometrico, ciò significa che la curva non presenta autointersezioni o cuspidi [5].

Esempio 3.1. *Consideriamo la curva*

$$E : y^2 = x^3 - x + 1.$$

Qui $a = -1$ e $b = 1$. Calcoliamo

$$4a^3 + 27b^2 = 4(-1)^3 + 27(1)^2 = -4 + 27 = 23.$$

Poiché $23 \neq 0$, la curva è non singolare e quindi può essere considerata una curva ellittica.

Esempio 3.2. *Consideriamo invece*

$$E : y^2 = x^3.$$

In questo caso $a = 0$ e $b = 0$, dunque

$$4a^3 + 27b^2 = 0.$$

La condizione di non singolarità non è soddisfatta. Infatti, la curva presenta una cuspide nell'origine e non è una curva ellittica valida per la struttura di gruppo che ci interessa.

3.5 Interpretazione geometrica

Sui numeri reali, una curva ellittica può essere rappresentata graficamente nel piano cartesiano. L'equazione

$$y^2 = x^3 + ax + b$$

implica che, per ogni valore di x , i possibili valori di y sono

$$y = \pm\sqrt{x^3 + ax + b},$$

quando il termine sotto radice è non negativo. Questo mostra una proprietà importante: se (x, y) è un punto della curva, allora anche $(x, -y)$ appartiene alla curva. La curva è quindi simmetrica rispetto all'asse delle ascisse [5].

$$(x, y) \in E \implies (x, -y) \in E. \quad (3.7)$$

Questa simmetria sarà fondamentale per definire l'inverso di un punto. Se $P = (x, y)$, il suo inverso sarà

$$-P = (x, -y). \quad (3.8)$$

Dal punto di vista geometrico, le curve ellittiche sui reali possono presentare forme diverse a seconda dei coefficienti a e b . In particolare, il grafico può avere una sola componente connessa oppure due componenti. Questa differenza dipende dal numero di radici reali del polinomio cubico $x^3 + ax + b$ [5].

3.6 Intersezione con le rette

Una proprietà geometrica centrale delle curve ellittiche è che una retta interseca una curva cubica in al più tre punti, contando anche molteplicità e punti all'infinito [5]. Nel caso delle curve ellittiche, questa proprietà permette di definire una legge di composizione tra punti.

Supponiamo che P e Q siano due punti distinti della curva. La retta passante per P e Q interseca la curva in un terzo punto, che indichiamo con R . La somma $P + Q$ sarà poi definita come il punto simmetrico di R rispetto all'asse delle ascisse.

In forma intuitiva:

1. si prende la retta passante per P e Q ;
2. si trova il terzo punto di intersezione R con la curva;
3. si riflette R rispetto all'asse x ;
4. il punto ottenuto è $P + Q$.

Questa costruzione sarà formalizzata nel Capitolo 4. Per ora è sufficiente osservare che la geometria della curva suggerisce naturalmente un'operazione binaria sui suoi punti.

3.7 Il punto all'infinito

Per definire una struttura di gruppo sui punti della curva, è necessario introdurre un elemento neutro. Questo elemento è il punto all'infinito, indicato con $\mathcal{O}$ [5].

Intuitivamente, $\mathcal{O}$ può essere pensato come il punto in cui si incontrano le rette verticali. Se $P = (x, y)$ è un punto della curva, il suo opposto è $-P = (x, -y)$. I due punti P e $-P$ sono allineati verticalmente. La retta verticale che passa per essi non incontra la curva in un terzo punto ordinario del piano reale; per completare la regola di intersezione, si introduce quindi il punto all'infinito.

In questo modo si può scrivere

$$P + (-P) = \mathcal{O}. \quad (3.9)$$

Il punto $\mathcal{O}$ agisce come elemento neutro della somma:

$$P + \mathcal{O} = \mathcal{O} + P = P. \quad (3.10)$$

Questa proprietà rende il punto all'infinito essenziale. Senza di esso, l'insieme dei punti ordinari della curva non sarebbe chiuso rispetto all'operazione di somma.

3.8 Perché partire dai numeri reali

Le curve ellittiche sui numeri reali non sono usate direttamente nei protocolli crittografici, perché richiederebbero calcoli con numeri reali e quindi introdurrebbero problemi di approssimazione. La crittografia richiede invece operazioni esatte, riproducibili e implementabili in modo efficiente. Per questo motivo, nei sistemi reali si lavora su campi finiti [5].

Tuttavia, il caso reale rimane utile per tre ragioni.

Primo, permette di visualizzare la curva. Concetti come simmetria, tangente, retta secante e punto all'infinito sono più comprensibili se introdotti geometricamente.

Secondo, permette di comprendere l'origine della legge di gruppo. La somma di punti non viene definita arbitrariamente: nasce dalla geometria delle intersezioni tra rette e curve cubiche.

Terzo, prepara il passaggio ai campi finiti. Quando lavoreremo su $\mathbb{F}_p$, non potremo più disegnare una curva continua, ma useremo formule algebriche che derivano esattamente dalla costruzione geometrica vista sui reali.

Capitolo 4

Legge di gruppo sulle curve ellittiche

4.1 Introduzione

Nel capitolo precedente abbiamo introdotto le curve ellittiche sui numeri reali come oggetti geometrici descritti da un'equazione del tipo

$$y^2 = x^3 + ax + b. \quad (4.1)$$

Il passo successivo è mostrare che i punti di una curva ellittica possono essere dotati di una struttura algebrica. Più precisamente, è possibile definire un'operazione di somma tra punti tale che l'insieme dei punti della curva, insieme al punto all'infinito $\mathcal{O}$, formi un gruppo abeliano [5, 6].

Questa osservazione è fondamentale per la crittografia su curve ellittiche. Infatti, i protocolli ECC non utilizzano la curva solo come figura geometrica, ma come insieme algebrico su cui è possibile calcolare espressioni del tipo

$$Q = kP,$$

dove P è un punto della curva e k è uno scalare intero. Per comprendere questa operazione, bisogna prima definire in modo preciso cosa significhi sommare due punti.

4.2 Idea geometrica della somma

Sia E una curva ellittica sui numeri reali e siano P e Q due punti di E . La somma $P + Q$ viene definita usando una costruzione geometrica basata sulle intersezioni tra rette e curve cubiche.

Se P e Q sono distinti, si considera la retta passante per P e Q . Questa retta interseca la curva in un terzo punto, che indichiamo con R . La somma $P + Q$ è definita come il punto simmetrico di R rispetto all'asse delle ascisse [5].

In simboli, se il terzo punto di intersezione è

$$R = (x_R, y_R),$$

allora

$$P + Q = -R = (x_R, -y_R).$$

Questa regola viene spesso descritta come metodo della “corda e tangente”. La corda è la retta che passa per due punti distinti della curva; la tangente compare invece nel caso in cui si voglia sommare un punto con se stesso.

4.3 Somma di due punti distinti

Consideriamo due punti distinti della curva

$$P = (x_P, y_P), \quad Q = (x_Q, y_Q),$$

con $P \neq Q$ e $Q \neq -P$. La retta passante per P e Q ha coefficiente angolare

$$\lambda = \frac{y_Q - y_P}{x_Q - x_P}. \quad (4.2)$$

La somma $S = P + Q$, con $S = (x_S, y_S)$, è data dalle formule [5, 6]

$$x_S = \lambda^2 - x_P - x_Q, \quad (4.3)$$

$$y_S = \lambda(x_P - x_S) - y_P. \quad (4.4)$$

Queste formule derivano dalla costruzione geometrica descritta prima. La retta interseca la curva in un terzo punto R ; poi il punto viene riflesso rispetto all’asse x , ottenendo $S = P + Q$.

Esempio 4.1. *Consideriamo la curva*

$$E : y^2 = x^3 - 7x + 10.$$

I punti

$$P = (1, 2), \quad Q = (3, 4)$$

appartengono alla curva, perché

$$2^2 = 1^3 - 7 \cdot 1 + 10 = 4$$

e

$$4^2 = 3^3 - 7 \cdot 3 + 10 = 16.$$

Calcoliamo ora $P + Q$. Il coefficiente angolare della retta passante per P e Q è

$$\lambda = \frac{4 - 2}{3 - 1} = 1.$$

Dunque

$$x_S = \lambda^2 - x_P - x_Q = 1^2 - 1 - 3 = -3,$$

e

$$y_S = \lambda(x_P - x_S) - y_P = 1(1 - (-3)) - 2 = 2.$$

Quindi

$$P + Q = (-3, 2).$$

Verifichiamo che il punto trovato appartiene alla curva:

$$2^2 = (-3)^3 - 7(-3) + 10 = -27 + 21 + 10 = 4.$$

Il risultato è quindi coerente.

4.4 Punti opposti e retta verticale

Un caso particolare si verifica quando i due punti hanno la stessa ascissa ma ordinata opposta. Se

$$P = (x, y),$$

allora il suo opposto è

$$-P = (x, -y).$$

La retta passante per P e $-P$ è verticale. Essa non interseca la curva in un terzo punto ordinario del piano reale; per completare la struttura algebrica, si introduce il punto all'infinito $\mathcal{O}$. Si definisce quindi

$$P + (-P) = \mathcal{O}. \quad (4.5)$$

Questa proprietà giustifica l'interpretazione di $-P$ come inverso additivo di P . In particolare, il punto all'infinito svolge il ruolo di elemento neutro rispetto alla somma [5].

4.5 Raddoppio di un punto

Il caso $P = Q$ richiede una costruzione leggermente diversa. Non esiste una retta passante per due punti distinti, perché i due punti coincidono. Si considera allora la tangente alla curva nel punto P .

Sia

$$P = (x_P, y_P)$$

un punto della curva, con $y_P \neq 0$. La pendenza della tangente in P è

$$\lambda = \frac{3x_P^2 + a}{2y_P}. \quad (4.6)$$

Il punto $2P = P + P$ ha coordinate

$$x_{2P} = \lambda^2 - 2x_P, \quad (4.7)$$

$$y_{2P} = \lambda(x_P - x_{2P}) - y_P. \quad (4.8)$$

Anche qui la logica è la stessa: la tangente interseca la curva in un ulteriore punto R , e il risultato $2P$ è il simmetrico di R rispetto all'asse delle ascisse [5, 6].

Esempio 4.2. *Consideriamo ancora la curva*

$$E : y^2 = x^3 - 7x + 10$$

e il punto

$$P = (1, 2).$$

In questo caso $a = -7$. Il coefficiente angolare della tangente è

$$\lambda = \frac{3x_P^2 + a}{2y_P} = \frac{3 \cdot 1^2 - 7}{2 \cdot 2} = \frac{-4}{4} = -1.$$

Quindi

$$x_{2P} = \lambda^2 - 2x_P = (-1)^2 - 2 = -1,$$

e

$$y_{2P} = \lambda(x_P - x_{2P}) - y_P = -1(1 - (-1)) - 2 = -4.$$

Pertanto

$$2P = (-1, -4).$$

Verifichiamo che il punto appartiene alla curva:

$$(-4)^2 = (-1)^3 - 7(-1) + 10 = -1 + 7 + 10 = 16.$$

Il risultato è quindi corretto.

4.6 Il caso della tangente verticale

Se $P = (x_P, y_P)$ e $y_P = 0$, la tangente alla curva in P è verticale. In questo caso il raddoppio del punto produce il punto all'infinito:

$$2P = \mathcal{O}. \quad (4.9)$$

Questo è coerente con la regola degli opposti. Infatti, se $y_P = 0$, allora

$$-P = (x_P, -y_P) = (x_P, 0) = P.$$

Quindi $P = -P$, e di conseguenza

$$P + P = P + (-P) = \mathcal{O}.$$

4.7 Elemento neutro

Il punto all'infinito $\mathcal{O}$ è l'elemento neutro della somma sui punti della curva [5]. Per ogni punto $P \in E$, si ha

$$P + \mathcal{O} = \mathcal{O} + P = P. \quad (4.10)$$

Questa proprietà è necessaria affinché l'insieme dei punti formi un gruppo. Senza un elemento neutro, la struttura non potrebbe soddisfare gli assiomi di gruppo introdotti nel Capitolo 2.

Dal punto di vista geometrico, il punto all'infinito può essere interpretato come il punto comune alle rette verticali nella chiusura proiettiva del piano. Nella trattazione crittografica, tuttavia, è sufficiente considerarlo come un elemento speciale che completa la legge di somma.

4.8 Inverso di un punto

Dato un punto

$$P = (x, y)$$

sulla curva, il suo inverso rispetto alla somma è

$$-P = (x, -y). \quad (4.11)$$

Infatti, P e $-P$ sono allineati verticalmente e la loro somma è il punto all'infinito:

$$P + (-P) = \mathcal{O}. \quad (4.12)$$

Inoltre, il punto all'infinito è inverso di se stesso:

$$-\mathcal{O} = \mathcal{O}. \quad (4.13)$$

Questa definizione è compatibile con la simmetria della curva rispetto all'asse delle ascisse: se (x, y) soddisfa l'equazione $y^2 = x^3 + ax + b$, allora anche $(x, -y)$ la soddisfa.

4.9 Proprietà di gruppo

Possiamo ora riassumere le proprietà principali della somma sui punti di una curva ellittica non singolare.

Sia

$$E(\mathbb{R}) = \{(x, y) \in \mathbb{R}^2 : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}.$$

Allora l'operazione di somma appena descritta soddisfa le proprietà di gruppo abeliano:

1. **Chiusura:** se $P, Q \in E(\mathbb{R})$, allora anche $P + Q \in E(\mathbb{R})$;

2. **Associatività:** per ogni $P, Q, R \in E(\mathbb{R})$,

$$(P + Q) + R = P + (Q + R);$$

3. **Elemento neutro:** esiste $\mathcal{O}$ tale che

$$P + \mathcal{O} = \mathcal{O} + P = P;$$

4. **Inverso:** per ogni $P \in E(\mathbb{R})$, esiste $-P \in E(\mathbb{R})$ tale che

$$P + (-P) = \mathcal{O};$$

5. **Commutatività:** per ogni $P, Q \in E(\mathbb{R})$,

$$P + Q = Q + P.$$

La commutatività è intuitiva dal punto di vista geometrico, perché la retta passante per P e Q è la stessa retta passante per Q e P . L'associatività è invece meno evidente geometricamente e richiede una dimostrazione più tecnica. In questa relazione ci limiteremo a usare il risultato: la legge di somma rende i punti di una curva ellittica non singolare un gruppo abeliano [5, 6].

4.10 Moltiplicazione scalare

Una volta definita la somma tra punti, è naturale definire la moltiplicazione di un punto per un intero. Se P è un punto della curva e k è un intero positivo, si definisce

$$kP = \underbrace{P + P + \cdots + P}_{k \text{ volte}}. \quad (4.14)$$

Per esempio,

$$2P = P + P,$$

$$3P = 2P + P,$$

$$4P = 2P + 2P.$$

Questa operazione prende il nome di moltiplicazione scalare. Essa è centrale nella crittografia su curve ellittiche. Nei protocolli ECC, uno scalare segreto k viene usato per calcolare un punto pubblico $Q = kP$, dove P è un punto base noto pubblicamente [5, 6].

La sicurezza si basa sul fatto che il calcolo diretto di $Q = kP$ è efficiente, mentre il problema inverso, cioè determinare k conoscendo P e Q , è computazionalmente difficile su curve e parametri scelti correttamente [5, 6].

4.11 Metodo double-and-add

Calcolare kP sommando P con se stesso k volte sarebbe inefficiente quando k è molto grande. In crittografia, gli scalari sono numeri di centinaia di bit, quindi è necessario usare algoritmi più efficienti.

Un metodo semplice è il metodo *double-and-add*, analogo all'esponenziazione veloce. L'idea è usare la rappresentazione binaria di k e combinare raddoppi e somme.

Per esempio, se

$$k = 13,$$

allora

$$13 = 8 + 4 + 1.$$

Quindi

$$13P = 8P + 4P + P.$$

I punti $2P, 4P, 8P$ si ottengono mediante raddoppi successivi:

$$2P, \quad 4P = 2(2P), \quad 8P = 2(4P).$$

Poi si sommano solo i multipli corrispondenti ai bit uguali a 1 nella rappresentazione binaria di k .

Algorithm 1 Moltiplicazione scalare double-and-add

Require: Punto P , scalare k

Ensure: Punto $Q = kP$

```

1:  $Q \leftarrow \mathcal{O}$ 
2:  $R \leftarrow P$ 
3: while  $k > 0$  do
4:   if  $k$  è dispari then
5:      $Q \leftarrow Q + R$ 
6:   end if
7:    $R \leftarrow 2R$ 
8:    $k \leftarrow \lfloor k/2 \rfloor$ 
9: end while
10: return  $Q$ 
```

Questo algoritmo mostra perché la moltiplicazione scalare è efficiente: il numero di operazioni cresce in modo proporzionale al numero di bit di k , non al valore numerico di k [6].

4.12 Importanza crittografica della legge di gruppo

La legge di gruppo sulle curve ellittiche è il fondamento di ECC. Senza un'operazione di somma ben definita, non sarebbe possibile parlare di multipli di un punto, né costruire il problema del logaritmo discreto su curve ellittiche.

Il passaggio concettuale è il seguente:

1. i punti della curva formano un gruppo abeliano;
2. in un gruppo si può ripetere l'operazione, definendo kP ;
3. il calcolo di kP può essere eseguito efficientemente;
4. il problema inverso, cioè recuperare k da P e kP , è difficile su curve scelte correttamente.

Questa struttura permette di costruire protocolli di scambio di chiavi e firme digitali. Nei prossimi capitoli vedremo come la stessa legge di gruppo viene trasferita dai numeri reali ai campi finiti, dove diventa adatta all'implementazione crittografica.

Capitolo 5

Curve ellittiche su campi finiti

5.1 Introduzione

Nei capitoli precedenti abbiamo studiato le curve ellittiche sui numeri reali per costruire l'intuizione geometrica della somma tra punti. Tuttavia, le curve sui reali non sono adatte all'uso crittografico, perché i protocolli richiedono operazioni esatte, riproducibili e implementabili in modo efficiente. I numeri reali introducono invece problemi di approssimazione e arrotondamento.

Per questo motivo, la crittografia su curve ellittiche lavora su campi finiti [5, 6]. Una scelta comune è definire la curva su un campo primo $\mathbb{F}_p$, dove p è un numero primo grande. Le coordinate dei punti appartengono quindi all'insieme

$$\mathbb{F}_p = \{0, 1, 2, \dots, p-1\},$$

e tutte le operazioni sono eseguite modulo p . La curva non appare più come una figura continua, ma come un insieme finito di punti. Nonostante ciò, la struttura algebrica rimane la stessa: si possono sommare punti, raddoppiare punti, definire il punto all'infinito e calcolare multipli scalari kP .

5.2 Curve ellittiche su $\mathbb{F}_p$

Sia $p > 3$ un numero primo. Una curva ellittica su $\mathbb{F}_p$ è definita da una congruenza della forma [5, 6]

$$y^2 \equiv x^3 + ax + b \pmod{p}, \quad (5.1)$$

dove $a, b \in \mathbb{F}_p$. L'insieme dei punti della curva è [5, 6]

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 \equiv x^3 + ax + b \pmod{p}\} \cup \{\mathcal{O}\}. \quad (5.2)$$

Il punto $\mathcal{O}$ è il punto all'infinito e svolge il ruolo di elemento neutro. Anche nel caso finito è necessario imporre la condizione di non singolarità [5]:

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p}. \quad (5.3)$$

Questa condizione evita degenerazioni che impedirebbero di definire correttamente la legge di gruppo.

5.3 Differenza rispetto al caso reale

Nel caso reale, una curva ellittica può essere disegnata come una curva continua nel piano cartesiano. Su $\mathbb{F}_p$, invece, le coordinate possibili sono soltanto p valori per x e p valori per y . Per esempio, su $\mathbb{F}_{17}$ le coordinate ammesse sono

$$0, 1, 2, \dots, 16.$$

Una curva ellittica su $\mathbb{F}_{17}$ non è quindi una curva liscia, ma un insieme discreto di punti [5]. La costruzione geometrica tramite rette e riflessioni fornisce l'intuizione, ma nei campi finiti l'operazione viene eseguita mediante formule algebriche modulari.

5.4 Esempio di curva su $\mathbb{F}_{17}$

Consideriamo la curva

$$E : y^2 \equiv x^3 + 2x + 2 \pmod{17}. \quad (5.4)$$

Per determinare i punti della curva, si controllano i valori $x \in \mathbb{F}_{17}$ e si verifica se

$$x^3 + 2x + 2 \pmod{17}$$

è un quadrato modulo 17. I quadrati modulo 17 sono:

y	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$y^2 \pmod{17}$	0	1	4	9	16	8	2	15	13	13	15	2	8	16	9	4	1

Per esempio, per $x = 0$ si ottiene

$$x^3 + 2x + 2 = 2,$$

e dalla tabella $y^2 \equiv 2 \pmod{17}$ per $y = 6$ e $y = 11$. Quindi $(0, 6)$ e $(0, 11)$ appartengono alla curva. Per $x = 3$,

$$3^3 + 2 \cdot 3 + 2 = 35 \equiv 1 \pmod{17},$$

da cui si ottengono i punti $(3, 1)$ e $(3, 16)$. Questo esempio mostra che una curva su campo finito è semplicemente l'insieme delle coppie che soddisfano una congruenza.

5.5 Somma di punti modulo p

La somma di punti su una curva ellittica su $\mathbb{F}_p$ usa le stesse formule del caso reale, ma ogni operazione viene eseguita modulo p [5, 6]. Siano

$$P = (x_P, y_P), \quad Q = (x_Q, y_Q),$$

con $P \neq Q$ e $Q \neq -P$. Il coefficiente angolare è

$$\lambda \equiv (y_Q - y_P)(x_Q - x_P)^{-1} \pmod{p}. \quad (5.5)$$

Le coordinate di $S = P + Q = (x_S, y_S)$ sono

$$x_S \equiv \lambda^2 - x_P - x_Q \pmod{p}, \quad (5.6)$$

$$y_S \equiv \lambda(x_P - x_S) - y_P \pmod{p}. \quad (5.7)$$

La divisione del caso reale viene quindi sostituita dalla moltiplicazione per l'inverso modulare.

Per esempio, sulla curva precedente con $P = (0, 6)$ e $Q = (3, 1)$,

$$\lambda \equiv (1 - 6)(3 - 0)^{-1} \equiv (-5) \cdot 3^{-1} \pmod{17}.$$

Poiché $3^{-1} \equiv 6 \pmod{17}$, si ha $\lambda \equiv 4 \pmod{17}$, e quindi

$$x_S \equiv 4^2 - 0 - 3 \equiv 13 \pmod{17}, \quad y_S \equiv 4(0 - 13) - 6 \equiv 10 \pmod{17},$$

da cui $P + Q = (13, 10)$, verificabile sostituendo nella congruenza della curva.

5.6 Raddoppio di un punto modulo p

Il raddoppio di un punto si ottiene adattando la formula della tangente al caso modulare [5, 6]. Se $P = (x_P, y_P)$ e $y_P \not\equiv 0 \pmod{p}$, allora

$$\lambda \equiv (3x_P^2 + a)(2y_P)^{-1} \pmod{p}. \quad (5.8)$$

Le coordinate di $2P$ sono

$$x_{2P} \equiv \lambda^2 - 2x_P \pmod{p}, \quad (5.9)$$

$$y_{2P} \equiv \lambda(x_P - x_{2P}) - y_P \pmod{p}. \quad (5.10)$$

Se $y_P \equiv 0 \pmod{p}$, la tangente è verticale e si pone

$$2P = \mathcal{O}.$$

Per esempio, sulla curva precedente e per $P = (0, 6)$, con $a = 2$,

$$\lambda \equiv (3 \cdot 0^2 + 2)(2 \cdot 6)^{-1} \equiv 2 \cdot 12^{-1} \pmod{17}.$$

Poiché $12^{-1} \equiv 10 \pmod{17}$, si ha

$$\lambda \equiv 20 \equiv 3 \pmod{17}.$$

Da cui

$$x_{2P} \equiv 3^2 - 0 \equiv 9 \pmod{17},$$

$$y_{2P} \equiv 3(0 - 9) - 6 \equiv -33 \equiv 1 \pmod{17}.$$

Quindi

$$2P = (9, 1).$$

5.7 Opposti e punto all'infinito

Su $\mathbb{F}_p$, l'opposto di un punto

$$P = (x, y)$$

è

$$-P = (x, -y \bmod p). \quad (5.11)$$

Se $y \neq 0$, questo può essere scritto anche come $(x, p - y)$ [5]. Per esempio, in $\mathbb{F}_{17}$, l'opposto di $(0, 6)$ è $(0, 11)$, perché

$$6 + 11 \equiv 0 \pmod{17}.$$

Come nel caso reale,

$$P + (-P) = \mathcal{O},$$

e il punto all'infinito rimane l'elemento neutro:

$$P + \mathcal{O} = \mathcal{O} + P = P.$$

5.8 Moltiplicazione scalare e sottogruppi ciclici

La moltiplicazione scalare su una curva ellittica su $\mathbb{F}_p$ è definita come [5, 6]

$$kP = \underbrace{P + P + \dots + P}_{k \text{ volte}}. \quad (5.12)$$

Ogni somma e ogni raddoppio vengono calcolati modulo p . Questa operazione è il nucleo computazionale di ECC.

Usando ancora la curva

$$E : y^2 \equiv x^3 + 2x + 2 \pmod{17}$$

e il punto $P = (0, 6)$, abbiamo già trovato

$$2P = (9, 1).$$

Calcoliamo $3P = 2P + P$. Sommando $(9, 1)$ e $(0, 6)$,

$$\lambda \equiv (6 - 1)(0 - 9)^{-1} \equiv 5 \cdot 8^{-1} \pmod{17}.$$

Poiché $8^{-1} \equiv 15 \pmod{17}$,

$$\lambda \equiv 75 \equiv 7 \pmod{17}.$$

Si ottiene

$$x_{3P} \equiv 7^2 - 9 - 0 \equiv 6 \pmod{17},$$

$$y_{3P} \equiv 7(9 - 6) - 1 \equiv 3 \pmod{17}.$$

Quindi

$$3P = (6, 3).$$

In crittografia si lavora spesso con un sottogruppo ciclico generato da un punto base G [5, 6]:

$$\langle G \rangle = \{G, 2G, 3G, \dots, nG = \mathcal{O}\}. \quad (5.13)$$

Il più piccolo intero positivo n tale che $nG = \mathcal{O}$ è chiamato ordine del punto G . Nei protocolli ECC, G è pubblico, mentre la chiave privata è uno scalare segreto d . La chiave pubblica è

$$Q = dG.$$

Il problema di sicurezza consiste nel recuperare d dati G e Q , cioè nel risolvere il logaritmo discreto su curve ellittiche [5, 6].

5.9 Perché i campi finiti sono adatti alla crittografia

Le curve ellittiche su campi finiti sono adatte alla crittografia perché tutte le operazioni sono esatte, gli elementi del campo hanno rappresentazioni finite e la struttura di gruppo consente di definire una funzione computazionalmente asimmetrica:

$$d \mapsto Q = dG.$$

Il calcolo di Q a partire da d e G è efficiente, mentre il recupero di d da G e Q è ritenuto difficile per parametri scelti correttamente. Questa combinazione di esattezza, finitezza e asimmetria computazionale rende le curve ellittiche su campi finiti particolarmente efficaci per la crittografia a chiave pubblica [5, 6].

Capitolo 6

Il problema del logaritmo discreto su curve ellittiche

6.1 Introduzione

Nei capitoli precedenti abbiamo costruito progressivamente la struttura matematica necessaria alla crittografia su curve ellittiche. Abbiamo visto che i punti di una curva ellittica su un campo finito formano un gruppo abeliano e che, fissato un punto base G , è possibile calcolare multipli del tipo [5, 6]

$$Q = dG,$$

dove d è un intero e Q è un altro punto della curva.

Questo capitolo studia il problema inverso: dato il punto base G e il punto $Q = dG$, quanto è difficile recuperare lo scalare d ? Questa domanda è centrale perché la sicurezza di molti protocolli basati su curve ellittiche dipende proprio dall'impraticabilità di tale inversione.

Il problema prende il nome di *Elliptic Curve Discrete Logarithm Problem*, abbreviato in ECDLP. Esso è l'analogo, nel gruppo dei punti di una curva ellittica, del problema del logaritmo discreto nei gruppi moltiplicativi modulo un primo [4, 5].

6.2 Richiamo: il logaritmo discreto classico

Nel caso classico, si considera un gruppo moltiplicativo modulo un numero primo p . Dato un generatore g e un esponente x , è efficiente calcolare

$$y \equiv g^x \pmod{p}. \quad (6.1)$$

Il problema inverso consiste nel trovare x conoscendo g , y e p . In altre parole, si vuole risolvere

$$y \equiv g^x \pmod{p}. \quad (6.2)$$

Il valore x , se esiste, è chiamato logaritmo discreto di y in base g .

L'importanza crittografica nasce dall'asimmetria computazionale: l'esponenziazione modulare è efficiente, mentre il calcolo del logaritmo discreto è ritenuto difficile in gruppi opportunamente scelti [4].

Esempio 6.1. Nel gruppo $\mathbb{Z}_{17}^*$, si ha

$$3^4 = 81 \equiv 13 \pmod{17}.$$

La direzione facile è calcolare 13 conoscendo 3 e 4. La direzione inversa è determinare 4 conoscendo 3 e 13, cioè risolvere

$$13 \equiv 3^x \pmod{17}.$$

Su un modulo piccolo si può procedere per tentativi. Su parametri crittografici, invece, questa ricerca diventa impraticabile.

6.3 Dal caso moltiplicativo al caso ellittico

La crittografia su curve ellittiche sostituisce il gruppo moltiplicativo modulo p con il gruppo additivo dei punti di una curva ellittica su un campo finito.

Nel caso moltiplicativo classico, l'operazione efficiente è

$$g^x \pmod{p}.$$

Nel caso ellittico, l'operazione corrispondente è la moltiplicazione scalare

$$Q = dG.$$

Qui:

- G è un punto base pubblico della curva;
- d è uno scalare intero, normalmente usato come chiave privata;
- Q è un punto della curva, normalmente usato come chiave pubblica [5, 6].

L'analogia può essere riassunta così:

Contesto	Operazione diretta	Problema inverso
Gruppo moltiplicativo	$y = g^x \bmod p$	trovare x da g e y
Curva ellittica	$Q = dG$	trovare d da G e Q

Tabella 6.1: Analogia tra logaritmo discreto classico e logaritmo discreto su curve ellittiche.

Nel primo caso l'operazione viene scritta in notazione moltiplicativa; nel secondo caso in notazione additiva. La differenza di notazione riflette la diversa operazione di gruppo, ma il principio crittografico è lo stesso: una direzione deve essere efficiente, l'altra deve essere computazionalmente difficile [4, 5].

6.4 Definizione di ECDLP

Sia $E(\mathbb{F}_p)$ una curva ellittica definita su un campo finito e sia G un punto della curva di ordine n . Consideriamo il sottogruppo ciclico generato da G :

$$\langle G \rangle = \{G, 2G, 3G, \dots, nG = \mathcal{O}\}. \quad (6.3)$$

Dato un punto $Q \in \langle G \rangle$, esiste un intero d , con $0 \leq d < n$, tale che

$$Q = dG. \quad (6.4)$$

Il problema del logaritmo discreto su curve ellittiche consiste nel determinare d conoscendo G , Q , la curva E e il campo finito su cui essa è definita [5, 6].

Definizione 6.1 (Elliptic Curve Discrete Logarithm Problem). *Siano $E(\mathbb{F}_p)$ una curva ellittica su un campo finito, $G \in E(\mathbb{F}_p)$ un punto di ordine n , e $Q \in \langle G \rangle$. Il problema del logaritmo discreto su curve ellittiche consiste nel trovare l'intero d , con $0 \leq d < n$, tale che*

$$Q = dG.$$

Il valore d può essere interpretato come il logaritmo discreto di Q in base G , ma nel gruppo additivo dei punti della curva.

6.5 Perché la direzione diretta è efficiente

Il calcolo diretto di

$$Q = dG$$

è efficiente perché può essere eseguito tramite algoritmi di moltiplicazione scalare come il metodo *double-and-add*. Tale metodo sfrutta la rappresentazione binaria di d e calcola dG mediante una sequenza di raddoppi e somme [6].

Se d ha m bit, il numero di operazioni richieste è proporzionale a m , non al valore numerico di d . Questo è essenziale: uno scalare di 256 bit può rappresentare un numero enorme, ma la sua lunghezza è soltanto 256 bit, e l'algoritmo lavora sulla lunghezza della rappresentazione.

Per esempio, se

$$d = 13,$$

allora

$$13 = 8 + 4 + 1,$$

e quindi

$$13G = 8G + 4G + G.$$

I punti $2G$, $4G$, $8G$ si ottengono con raddoppi successivi, e poi si sommano i punti corrispondenti ai bit uguali a 1.

Questa efficienza rende possibile usare ECC in protocolli reali: generare una chiave pubblica a partire da una chiave privata deve essere un'operazione rapida per l'utente legittimo.

6.6 Perché l'inversione è difficile

Il problema inverso è molto diverso. Dati G e

$$Q = dG,$$

un avversario vuole recuperare d . Il metodo più immediato è la forza bruta: calcolare

$$G, 2G, 3G, \dots$$

fino a trovare Q . Se $Q = kG$, allora l'avversario ha trovato $k = d$.

Questo metodo è corretto, ma diventa impraticabile quando l'ordine n del punto base è molto grande. Se n è dell'ordine di 2^{256} , una ricerca esaustiva richiederebbe, nel caso peggiore, un numero astronomico di operazioni.

La sicurezza di ECC dipende quindi dalla scelta di parametri tali che:

- il punto base G abbia ordine grande;
- il sottogruppo generato da G non presenti strutture deboli;
- non siano noti algoritmi efficienti per risolvere l'ECDLP su quella curva.

In questo senso, ECC non si basa sul fatto che l'inversione sia logicamente impossibile, ma sul fatto che essa sia computazionalmente impraticabile con le risorse disponibili [4, 6].

6.7 Esempio su una curva piccola

Per comprendere il problema, consideriamo ancora la curva

$$E : y^2 \equiv x^3 + 2x + 2 \pmod{17}$$

e il punto

$$G = (0, 6).$$

Nel capitolo precedente abbiamo calcolato:

$$2G = (9, 1),$$

e

$$3G = (6, 3).$$

Supponiamo ora che un osservatore conosca G e il punto

$$Q = (6, 3).$$

Su questa curva piccola, può provare i multipli di G :

$$G = (0, 6),$$

$$2G = (9, 1),$$

$$3G = (6, 3).$$

A questo punto conclude che

$$Q = 3G,$$

quindi il logaritmo discreto di Q in base G è

$$d = 3.$$

Questo esempio mostra la natura del problema, ma non la sua difficoltà reale. Con parametri piccoli, l'ECDLP è facilmente risolvibile per tentativi. La sicurezza nasce quando il sottogruppo generato dal punto base ha ordine enorme.

6.8 Ordine del punto e dimensione del sottogruppo

L'ordine del punto base G è un parametro cruciale. Per definizione, l'ordine di G è il più piccolo intero positivo n tale che

$$nG = \mathcal{O}. \tag{6.5}$$

Il sottogruppo generato da G contiene quindi esattamente n elementi:

$$\langle G \rangle = \{\mathcal{O}, G, 2G, \dots, (n-1)G\}.$$

Se n è piccolo, l'avversario può enumerare tutti i multipli di G e risolvere il problema. Se invece n è molto grande, l'enumerazione diventa impraticabile.

Per questa ragione, nei protocolli ECC il punto base viene scelto con ordine grande, spesso un numero primo o con un grande fattore primo. Questo riduce la possibilità di attacchi basati sulla decomposizione del gruppo in sottogruppi più piccoli [5, 6].

6.9 Forza bruta e attacchi generici

L'attacco più semplice contro ECDLP è la ricerca esaustiva. L'avversario calcola successivamente i multipli di G fino a raggiungere Q . Il costo di questo metodo è lineare nell'ordine del sottogruppo, quindi dell'ordine di n .

Esistono però attacchi generici migliori della forza bruta semplice, come il metodo baby-step giant-step e il metodo rho di Pollard. Questi algoritmi non sfruttano debolezze specifiche della curva, ma lavorano in modo generale sui gruppi.

In particolare, molti attacchi generici hanno complessità dell'ordine di circa

$$O(\sqrt{n}). \quad (6.6)$$

Questo significa che, per ottenere un livello di sicurezza di circa 2^{128} operazioni contro attacchi generici, si scelgono sottogruppi di ordine approssimativamente 2^{256} . Questa è una delle ragioni per cui molte curve moderne usano parametri intorno ai 256 bit [7, 6].

Questa osservazione collega direttamente la dimensione del gruppo alla sicurezza: non basta scegliere una curva qualsiasi; bisogna scegliere una curva e un punto base con parametri adeguati.

6.10 ECDLP come funzione one-way

La funzione

$$d \mapsto Q = dG \quad (6.7)$$

può essere vista come una funzione one-way nel contesto del gruppo dei punti della curva. Essa è facile da calcolare: dato d , si ottiene Q in modo efficiente tramite moltiplicazione scalare. Tuttavia, è difficile da invertire: dato Q , recuperare d richiede risolvere l'ECDLP [5, 4, 6].

Questa funzione non è una funzione one-way con trap-door nello stesso senso di RSA. In RSA, esiste una informazione segreta, collegata alla fattorizzazione del modulo, che permette di invertire efficientemente la funzione. Nel caso tipico di ECC, invece, la sicurezza dei protocolli deriva dal fatto che solo il proprietario conosce lo scalare privato d ; non esiste una scorciatoia pubblica per recuperarlo da $Q = dG$.

Questa distinzione è importante. ECC non richiede di nascondere la curva, il campo o il punto base. Al contrario, questi parametri sono normalmente pubblici. Ciò che deve rimanere segreto è lo scalare d .

6.11 Ruolo di ECDLP nella crittografia a chiave pubblica

L'ECDLP permette di costruire coppie di chiavi asimmetriche. Il procedimento generale è il seguente:

1. si scelgono pubblicamente una curva E , un campo finito $\mathbb{F}_p$, un punto base G e l'ordine n di G ;
2. l'utente sceglie casualmente uno scalare privato d , con $1 \leq d < n$;
3. calcola la chiave pubblica

$$Q = dG;$$

4. pubblica Q , mantenendo segreto d .

Un avversario può conoscere E , $\mathbb{F}_p$, G , n e Q . Tuttavia, per ricavare d , dovrebbe risolvere il logaritmo discreto di Q in base G . La sicurezza della chiave privata dipende quindi direttamente dalla difficoltà dell'ECDLP.

Questa struttura verrà usata nei capitoli successivi per descrivere protocolli come ECDH ed ECDSA. In entrambi i casi, la chiave privata è uno scalare, mentre la chiave pubblica è un punto della curva [5, 6].

6.12 Confronto concettuale con RSA

È utile confrontare ECC con RSA, perché entrambi sono sistemi a chiave pubblica, ma si basano su problemi matematici diversi.

In RSA, la sicurezza è legata alla difficoltà di fattorizzare un grande intero $n = pq$, prodotto di due primi segreti. Conoscendo la fattorizzazione, è possibile calcolare la chiave privata; senza di essa, l'inversione è ritenuta difficile.

In ECC, invece, la sicurezza è legata alla difficoltà di recuperare lo scalare d da

$$Q = dG.$$

Quindi:

Sistema	Operazione facile	Problema difficile
RSA	moltiplicare p e q	fattorizzare $n = pq$
Diffie–Hellman classico	calcolare $g^x \bmod p$	logaritmo discreto modulo p
ECC	calcolare dG	logaritmo discreto su curve ellittiche

Tabella 6.2: Problemi computazionali alla base di diversi sistemi a chiave pubblica.

Questa differenza spiega perché la sicurezza dei vari sistemi deve essere valutata rispetto al problema matematico specifico su cui si fondano [12, 7].

6.13 Limite quantistico

La difficoltà dell'ECDLP è una difficoltà classica: riguarda gli algoritmi noti su calcolatori tradizionali. Il quadro cambia se si considerano computer quantistici sufficientemente potenti.

L'algoritmo di Shor mostra che, su un computer quantistico fault-tolerant sufficientemente grande, problemi come la fattorizzazione di interi, il logaritmo discreto nei gruppi finiti e il logaritmo discreto su curve ellittiche possono essere risolti in tempo polinomiale [13].

Questo significa che ECC, come RSA e Diffie–Hellman classico, non è considerata sicura in uno scenario post-quantistico. Tale limite non invalida l'importanza attuale di ECC, ma spiega perché la ricerca crittografica moderna stia sviluppando schemi alternativi resistenti ad attacchi quantistici [13].

Questo tema verrà ripreso nel capitolo dedicato ai limiti e alla prospettiva post-quantistica.

Capitolo 7

Protocolli crittografici basati su ECC

7.1 Introduzione

Nei capitoli precedenti abbiamo definito la struttura matematica di ECC: curve ellittiche su campi finiti, legge di gruppo, moltiplicazione scalare e problema del logaritmo discreto su curve ellittiche. Questo capitolo mostra come tali strumenti vengano usati in protocolli crittografici concreti.

L'idea centrale è la seguente: dato un punto base pubblico G , uno scalare segreto d permette di calcolare efficientemente

$$Q = dG.$$

Lo scalare d costituisce la chiave privata, mentre il punto Q costituisce la chiave pubblica [5, 6]. La sicurezza dipende dal fatto che, conoscendo G e Q , un avversario non riesca a recuperare d in tempo praticabile.

I tre usi principali trattati in questo capitolo sono:

1. generazione delle chiavi ECC;
2. scambio di chiavi tramite ECDH/ECDHE;
3. firme digitali tramite ECDSA.

7.2 Parametri pubblici e generazione delle chiavi

Un sistema ECC su un campo primo è descritto da parametri pubblici [5, 6]:

$$(E, \mathbb{F}_p, G, n),$$

dove E è la curva ellittica, $\mathbb{F}_p$ è il campo finito, G è il punto base e n è l'ordine di G , cioè il più piccolo intero positivo tale che

$$nG = \mathcal{O}.$$

Questi parametri non devono essere segreti. Nei sistemi reali sono normalmente standardizzati e pubblici. Ciò che deve rimanere segreto è lo scalare privato dell'utente.

La generazione di una coppia di chiavi ECC avviene in due passi [5, 6]:

1. si sceglie casualmente una chiave privata

$$d \in \{1, \dots, n-1\};$$

2. si calcola la chiave pubblica

$$Q = dG.$$

La scelta di d deve essere casuale e uniforme. Se la generazione casuale è debole o prevedibile, la sicurezza può essere compromessa indipendentemente dalla difficoltà matematica dell'ECDLP.

7.3 Elliptic Curve Diffie–Hellman

Elliptic Curve Diffie–Hellman, abbreviato ECDH, è la versione su curve ellittiche del protocollo Diffie–Hellman [10, 6]. Il suo obiettivo è permettere a due parti, Alice e Bob, di ottenere una chiave segreta condivisa comunicando su un canale insicuro.

Alice sceglie una chiave privata d_A e calcola

$$Q_A = d_A G.$$

Bob sceglie una chiave privata d_B e calcola

$$Q_B = d_B G.$$

Le chiavi pubbliche Q_A e Q_B vengono scambiate sul canale pubblico. Alice calcola

$$S_A = d_A Q_B = d_A (d_B G) = d_A d_B G,$$

mentre Bob calcola

$$S_B = d_B Q_A = d_B (d_A G) = d_B d_A G.$$

Poiché la moltiplicazione tra interi è commutativa, si ha

$$S_A = S_B = S.$$

Alice e Bob ottengono quindi lo stesso punto segreto S , senza trasmetterlo direttamente [10, 6].

Il punto S non viene usato direttamente come chiave simmetrica. Se

$$S = (x_S, y_S),$$

normalmente si applica una funzione di derivazione della chiave [6]:

$$K = \text{KDF}(x_S).$$

La chiave K può poi essere usata con un cifrario simmetrico o con una modalità autenticata.

7.4 Sicurezza e autenticazione in ECDH

Un avversario passivo che osserva il canale pubblico conosce

$$E, \mathbb{F}_p, G, n, Q_A, Q_B.$$

Per ricavare il segreto condiviso dovrebbe recuperare d_A da $Q_A = d_A G$, oppure d_B da $Q_B = d_B G$. Questo richiede la risoluzione dell'ECDLP.

ECDH, però, nella sua forma base non autentica le parti. Se le chiavi pubbliche scambiate non vengono verificate, il protocollo è vulnerabile a un attacco man-in-the-middle. Un avversario può sostituire Q_A e Q_B con proprie chiavi pubbliche, stabilendo due segreti distinti: uno con Alice e uno con Bob.

Per questo motivo, nei protocolli reali ECDH viene combinato con meccanismi di autenticazione, come certificati digitali o firme digitali [10, 14]. Il problema non è solo ottenere un segreto condiviso, ma sapere con chi lo si sta condividendo.

7.5 ECDHE e forward secrecy

Una variante importante è ECDHE, cioè *Elliptic Curve Diffie–Hellman Ephemeral*. In questo caso le chiavi usate per lo scambio sono temporanee e vengono generate per una singola sessione.

L'uso di chiavi effimere permette di ottenere *forward secrecy* [14, 6]: se in futuro una chiave privata a lungo termine viene compromessa, le chiavi di sessione passate non vengono automaticamente compromesse. Per questa ragione ECDHE è ampiamente usato nei protocolli moderni di comunicazione sicura.

7.6 Firme digitali su curve ellittiche

Oltre allo scambio di chiavi, ECC viene usata per costruire firme digitali. Una firma digitale garantisce autenticità, integrità e non ripudio [2]: il destinatario può verificare che il messaggio provenga dal possessore della chiave privata, che non sia stato modificato e che il firmatario non possa ragionevolmente negare la firma.

Lo schema più noto basato su curve ellittiche è ECDSA, *Elliptic Curve Digital Signature Algorithm* [15, 6]. Un utente possiede una chiave privata d e una chiave pubblica

$$Q = dG.$$

Per firmare un messaggio m , non si firma direttamente l'intero messaggio, ma il suo hash [2, 15]:

$$e = H(m).$$

La firma prodotta è una coppia

$$(r, s).$$

Chiunque conosca m , (r, s) , Q e i parametri della curva può verificare la validità della firma.

7.7 Generazione e verifica di una firma ECDSA

Siano pubblici i parametri $(E, \mathbb{F}_p, G, n)$, con G di ordine n . Sia d la chiave privata del firmatario e $Q = dG$ la chiave pubblica.

Per firmare un messaggio m , la generazione della firma ECDSA segue i passaggi seguenti [15, 6]:

1. si calcola

$$e = H(m);$$

2. si sceglie un nonce segreto k , con $1 \leq k < n$;

3. si calcola

$$R = kG = (x_R, y_R);$$

4. si pone

$$r \equiv x_R \pmod{n};$$

5. si calcola

$$s \equiv k^{-1}(e + dr) \pmod{n}.$$

La firma è la coppia (r, s) .

Per verificare la firma, il verificatore usa la chiave pubblica Q e calcola [15, 6]:

$$w \equiv s^{-1} \pmod{n},$$

$$u_1 \equiv ew \pmod{n}, \quad u_2 \equiv rw \pmod{n},$$

e poi il punto

$$X = u_1G + u_2Q.$$

La firma viene accettata se

$$r \equiv x_X \pmod{n},$$

dove x_X è la coordinata x del punto X .

La correttezza deriva dal fatto che, sostituendo $Q = dG$, si ottiene

$$u_1G + u_2Q = ewG + rw(dG) = (e + rd)wG.$$

Poiché

$$s \equiv k^{-1}(e + dr) \pmod{n},$$

si ha

$$w = s^{-1} \equiv (e + dr)^{-1}k \pmod{n},$$

e quindi il punto calcolato dal verificatore coincide con kG , cioè con il punto usato dal firmatario per costruire r .

7.8 Importanza del nonce in ECDSA

Il nonce k è critico per la sicurezza di ECDSA [15]. Deve essere segreto, imprevedibile e usato una sola volta. Se lo stesso k viene riutilizzato per firmare due messaggi diversi, la chiave privata può essere recuperata.

Infatti, per due hash e_1 ed e_2 firmati con lo stesso nonce, si hanno

$$s_1 \equiv k^{-1}(e_1 + dr) \pmod{n},$$

$$s_2 \equiv k^{-1}(e_2 + dr) \pmod{n}.$$

Sottraendo,

$$s_1 - s_2 \equiv k^{-1}(e_1 - e_2) \pmod{n},$$

da cui

$$k \equiv (e_1 - e_2)(s_1 - s_2)^{-1} \pmod{n}.$$

Una volta noto k , dalla formula

$$s \equiv k^{-1}(e + dr) \pmod{n}$$

si ricava

$$d \equiv (sk - e)r^{-1} \pmod{n}.$$

Questo mostra che la sicurezza di ECDSA non dipende soltanto dall'ECDLP, ma anche dall'uso corretto dei valori temporanei.

7.9 ECDH ed ECDSA a confronto

ECDH ed ECDSA usano la stessa struttura matematica, ma con obiettivi diversi.

Protocollo	Obiettivo	Uso della chiave privata
ECDH	stabilire una chiave condivisa	combinata con la chiave pubblica altrui
ECDSA	firmare un messaggio	usata per produrre una firma verificabile

Tabella 7.1: Confronto tra ECDH ed ECDSA.

Nel primo caso, due parti contribuiscono alla generazione di un segreto comune. Nel secondo, una parte produce una firma che chiunque può verificare usando la chiave pubblica. In entrambi i casi, la sicurezza dipende dalla difficoltà di ricavare uno scalare privato da un punto pubblico della forma $Q = dG$ [5, 6].

Capitolo 8

Applicazioni moderne di ECC

8.1 Introduzione

La crittografia su curve ellittiche non è soltanto una costruzione matematica astratta, ma una tecnologia utilizzata in protocolli e sistemi reali. La sua importanza pratica deriva dal fatto che permette di realizzare operazioni di crittografia a chiave pubblica con chiavi relativamente brevi e buone prestazioni computazionali. Per questo motivo ECC è impiegata in protocolli di comunicazione sicura, firme digitali, infrastrutture a chiave pubblica e sistemi blockchain [14, 8, 6].

In questo capitolo vengono considerate due applicazioni principali: l'uso di ECC nei protocolli di comunicazione sicura, in particolare tramite ECDHE, e l'uso di ECC in Bitcoin, dove le curve ellittiche sono alla base della generazione delle chiavi, degli indirizzi e della firma delle transazioni.

8.2 ECC nei protocolli di comunicazione sicura

Quando un utente accede a un servizio online, come un sito di e-commerce o una piattaforma bancaria, client e server devono stabilire un canale sicuro. Tale canale deve garantire confidenzialità, integrità e autenticazione. Per ottenere queste proprietà, i protocolli moderni combinano più strumenti crittografici: scambio di chiavi, cifratura simmetrica, funzioni hash, MAC, certificati digitali e firme.

ECC entra in questo contesto soprattutto attraverso ECDHE, cioè *Elliptic Curve Diffie–Hellman Ephemeral* [14, 6]. Questo protocollo permette a client e server di generare una chiave di sessione condivisa senza trasmetterla direttamente sul canale.

In forma semplificata, client e server concordano una curva ellittica e un punto base G . Il client sceglie uno scalare temporaneo a e invia

$$A = aG.$$

Il server sceglie uno scalare temporaneo b e invia

$$B = bG.$$

Il client calcola

$$S_C = aB = a(bG) = abG,$$

mentre il server calcola

$$S_S = bA = b(aG) = abG.$$

Entrambe le parti ottengono quindi lo stesso punto segreto [14, 6]:

$$S = abG.$$

Da questo punto viene poi derivata una chiave simmetrica, normalmente tramite una funzione di derivazione della chiave [6]:

$$K = \text{KDF}(x_S),$$

dove x_S è una coordinata del punto condiviso. La chiave K viene usata per cifrare e autenticare i dati della sessione con algoritmi simmetrici.

8.3 Forward secrecy e autenticazione

Il termine *ephemeral* indica che gli scalari usati nello scambio vengono generati per una singola sessione e poi eliminati. Questo consente di ottenere *forward secrecy* [14]: anche se in futuro una chiave privata a lungo termine venisse compromessa, le sessioni passate non sarebbero automaticamente decifrabili.

ECDHE da solo, però, non autentica le parti. Senza autenticazione, un avversario potrebbe eseguire un attacco man-in-the-middle, sostituendo le chiavi pubbliche scambiate e stabilendo due segreti distinti: uno con il client e uno con il server. Per evitare questo problema, nei protocolli reali lo scambio di chiavi viene autenticato tramite certificati digitali o firme.

Il certificato del server lega una chiave pubblica alla sua identità, attraverso la firma di una Certification Authority fidata [14]. In questo modo, il client può verificare di comunicare con il server corretto e non con un intermediario malevolo.

8.4 Crittografia ibrida

Nei protocolli moderni, la crittografia a chiave pubblica non viene usata per cifrare direttamente grandi quantità di dati. Le operazioni asimmetriche sono più costose delle operazioni simmetriche. Per questo motivo, si usa un approccio ibrido [14]:

1. ECDHE stabilisce un segreto condiviso;
2. da questo segreto vengono derivate chiavi simmetriche;
3. i dati applicativi vengono cifrati con algoritmi simmetrici efficienti;
4. l'integrità viene garantita tramite MAC o modalità di cifratura autenticata.

ECC risolve quindi il problema dello scambio sicuro della chiave, mentre la cifratura simmetrica protegge il traffico effettivo.

8.5 ECC in Bitcoin

Un'altra applicazione importante di ECC è Bitcoin [8, 16]. In Bitcoin, la crittografia su curve ellittiche serve principalmente a dimostrare il diritto di spendere determinati fondi. Un wallet non contiene materialmente bitcoin, ma gestisce chiavi private che permettono di firmare transazioni.

Il meccanismo fondamentale è:

chiave privata $\longrightarrow$ chiave pubblica $\longrightarrow$ indirizzo.

La chiave privata è uno scalare segreto d . La chiave pubblica corrispondente è ottenuta tramite moltiplicazione scalare:

$$Q = dG.$$

La rete può verificare firme prodotte con la chiave privata usando la chiave pubblica, ma non può ricavare la chiave privata da Q , a meno di risolvere l'ECDLP [8, 6].

8.6 La curva secp256k1

Bitcoin utilizza la curva ellittica secp256k1 [8, 9], definita su un campo primo $\mathbb{F}_p$, dove

$$p = 2^{256} - 2^{32} - 977.$$

L'equazione della curva è

$$y^2 \equiv x^3 + 7 \pmod{p}.$$

In questa curva i coefficienti sono

$$a = 0, \quad b = 7.$$

Dato il punto base G , una chiave privata Bitcoin è uno scalare d , mentre la chiave pubblica è

$$Q = dG.$$

Questa è esattamente la stessa struttura studiata nei capitoli precedenti: il calcolo diretto è efficiente, mentre il recupero di d da G e Q è computazionalmente difficile.

8.7 Wallet, seed phrase e indirizzi

Un wallet Bitcoin gestisce chiavi private, chiavi pubbliche e indirizzi [8]. Nei wallet moderni, le chiavi vengono spesso derivate da una *seed phrase*, cioè una sequenza di parole che permette di ricostruire l'intero wallet.

In forma semplificata:

seed phrase $\longrightarrow$ seed binaria $\longrightarrow$ chiavi private $\longrightarrow$ chiavi pubbliche $\longrightarrow$ indirizzi.

La seed phrase è quindi estremamente sensibile: chiunque la conosca può ricostruire le chiavi private e spendere i fondi associati.

Un indirizzo Bitcoin non coincide direttamente con la chiave pubblica, ma viene derivato da essa tramite funzioni hash e codifiche [8]. In forma semplificata:

$$Q \longrightarrow \text{SHA-256}(Q) \longrightarrow \text{RIPEMD-160}(\text{SHA-256}(Q)) \longrightarrow \text{indirizzo}.$$

L'indirizzo può essere condiviso pubblicamente per ricevere pagamenti, mentre la chiave privata deve rimanere segreta.

8.8 Transazioni, UTXO e firme

Bitcoin utilizza un modello basato su UTXO, cioè *Unspent Transaction Output* [8]. Un UTXO è un output di una transazione precedente che non è ancora stato speso. Quando un utente effettua un pagamento, il wallet seleziona uno o più UTXO e li usa come input di una nuova transazione.

Per spendere un UTXO, il wallet deve dimostrare di conoscere la chiave privata associata alla condizione di spesa. Questa dimostrazione avviene tramite una firma digitale. In forma semplificata:

1. il wallet costruisce la transazione;
2. calcola un hash dei dati da firmare;
3. firma tale hash con la chiave privata;
4. inserisce nella transazione la firma e le informazioni necessarie alla verifica.

Quando un nodo riceve la transazione, verifica che gli input facciano riferimento a UTXO esistenti e non ancora spesi, che la somma degli input sia sufficiente e che le firme siano valide. Se la firma è corretta, il nodo conclude che la spesa è stata autorizzata dal possessore della chiave privata corrispondente [8].

Dopo la firma, la transazione viene trasmessa alla rete tramite broadcast. I nodi la verificano e, se valida, la inseriscono nella mempool. Successivamente, un miner può includerla in un blocco. Quando il blocco viene accettato nella catena principale, la transazione riceve una conferma [8].

8.9 ECC e decentralizzazione

Il ruolo di ECC in Bitcoin è strettamente legato alla decentralizzazione. In un sistema bancario tradizionale, l'autorizzazione di un pagamento dipende da un intermediario centrale. In Bitcoin, invece, l'autorizzazione è verificabile crittograficamente.

Chi possiede la chiave privata può produrre una firma valida. Chiunque conosca la chiave pubblica o la condizione di spesa può verificare tale firma. In questo modo, i nodi della rete possono controllare autonomamente se una transazione è autorizzata, senza conoscere l'identità reale dell'utente e senza affidarsi a un'autorità centrale [8, 16].

8.10 Limiti pratici

Le applicazioni descritte mostrano l'efficacia di ECC, ma anche la necessità di implementazioni corrette. La sicurezza non dipende solo dalla difficoltà matematica dell'ECDLP, ma anche dalla qualità della generazione casuale, dalla protezione delle chiavi e dalla corretta validazione dei dati [6].

Tra i principali rischi pratici vi sono:

- generazione debole delle chiavi private;
- riuso o prevedibilità del nonce nelle firme digitali;
- mancata validazione delle chiavi pubbliche ricevute;
- scelta di curve o parametri non adeguati;
- vulnerabilità side-channel.

Questi rischi mostrano che una costruzione matematica sicura può diventare vulnerabile se implementata male. Per questo motivo, nei sistemi reali si usano librerie crittografiche mature, standard riconosciuti e procedure rigorose di validazione.

Capitolo 9

Confronto, limiti e prospettiva post-quantistica

9.1 Introduzione

Dopo aver analizzato le basi matematiche di ECC, i protocolli principali e alcune applicazioni moderne, è utile valutare la crittografia su curve ellittiche in modo critico. ECC è una delle tecniche più importanti della crittografia a chiave pubblica, soprattutto perché permette di ottenere livelli di sicurezza elevati con chiavi più brevi rispetto a sistemi come RSA. Tuttavia, questa efficienza non elimina i limiti: la sicurezza dipende dalla scelta dei parametri, dalla qualità dell'implementazione, dalla generazione corretta dei valori casuali e, in prospettiva futura, dalla resistenza rispetto agli attacchi quantistici [5, 7, 6].

Questo capitolo confronta ECC con RSA, discute i principali limiti pratici e teorici delle curve ellittiche, e introduce la prospettiva post-quantistica.

9.2 ECC e RSA

RSA ed ECC sono entrambi sistemi a chiave pubblica, ma si basano su problemi matematici differenti. RSA si fonda sulla difficoltà di fattorizzare un grande intero n , prodotto di due numeri primi segreti:

$$n = pq.$$

Moltiplicare p e q è efficiente; recuperare p e q conoscendo soltanto n è ritenuto difficile per numeri sufficientemente grandi [12].

ECC, invece, si basa sul problema del logaritmo discreto su curve ellittiche. Dato un punto base G e uno scalare segreto d , è efficiente calcolare

$$Q = dG.$$

Il problema difficile consiste nel recuperare d conoscendo soltanto G e Q [5, 6]. La differenza può essere riassunta così:

Il principio comune è l'asimmetria computazionale: l'utente legittimo deve poter eseguire facilmente l'operazione diretta, mentre l'avversario deve trovarsi davanti a un problema inverso impraticabile [12, 10, 5].

Sistema	Operazione facile	Problema difficile
RSA	Calcolare $n = pq$	Fattorizzare n
Diffie–Hellman classico	Calcolare $g^x \bmod p$	Logaritmo discreto modulo p
ECC	Calcolare $Q = dG$	Logaritmo discreto su curve ellittiche

Tabella 9.1: Problemi computazionali alla base di alcuni sistemi a chiave pubblica.

9.3 Efficienza e dimensione delle chiavi

Uno dei principali vantaggi pratici di ECC è la dimensione ridotta delle chiavi. A parità di sicurezza stimata, ECC richiede chiavi molto più brevi rispetto a RSA. Questo riduce memoria, larghezza di banda e dimensione dei certificati [7, 6].

Sicurezza stimata	RSA	ECC
Circa 112 bit	2048 bit	224 bit
Circa 128 bit	3072 bit	256 bit
Circa 192 bit	7680 bit	384 bit
Circa 256 bit	15360 bit	512 bit

Tabella 9.2: Confronto indicativo tra dimensioni delle chiavi RSA ed ECC.

Questa caratteristica rende ECC adatta a protocolli di rete, dispositivi mobili, smart card, sistemi embedded e applicazioni blockchain [7, 6]. Chiavi più brevi non significano però sicurezza automatica: la robustezza dipende dalla scelta della curva, dall'ordine del sottogruppo, dalla corretta generazione delle chiavi e dalla qualità dell'implementazione.

9.4 Limiti teorici

ECC non offre sicurezza assoluta. Come gran parte della crittografia moderna, si basa su un'assunzione computazionale: l'ECDLP è ritenuto difficile per curve e parametri scelti correttamente. Non esiste però una dimostrazione matematica definitiva che un algoritmo classico efficiente per risolvere l'ECDLP non possa esistere [4, 6].

Questo non rende ECC insicura; descrive semplicemente la natura della sicurezza computazionale. Un sistema crittografico è considerato sicuro se resiste ai migliori attacchi noti e se il costo dell'attacco rimane fuori dalla portata di un avversario realistico.

La formulazione corretta è quindi:

ECC è sicura finché il problema del logaritmo discreto su curve ellittiche rimane computazionalmente impraticabile per i parametri utilizzati.

9.5 Limiti implementativi

Molti attacchi contro sistemi crittografici non rompono direttamente il problema matematico sottostante, ma sfruttano errori di implementazione. ECC non fa eccezione.

Un primo rischio riguarda la generazione delle chiavi private. Lo scalare segreto d deve essere generato in modo casuale e uniforme nell'intervallo corretto. Se il generatore di numeri casuali è prevedibile, l'avversario può restringere lo spazio delle possibili chiavi.

Un secondo rischio riguarda le firme digitali. In schemi come ECDSA, il nonce temporaneo k deve essere segreto, casuale e usato una sola volta. Se lo stesso nonce viene riutilizzato per firmare due messaggi diversi, la chiave privata può essere recuperata algebricamente. In questo caso non viene risolto l'ECDLP: la chiave viene esposta a causa di un errore nell'uso del protocollo [2, 15].

Un terzo rischio riguarda la validazione delle chiavi pubbliche. Quando un sistema riceve un punto da un'altra parte, deve verificare che appartenga alla curva corretta e al sottogruppo previsto. In assenza di questi controlli, possono comparire attacchi basati su punti non validi o sottogruppi piccoli [6].

Infine, esistono attacchi side-channel, che sfruttano informazioni indirette come tempi di esecuzione, consumo energetico, comportamento della cache o emissioni elettromagnetiche. Per ridurre questi rischi, le implementazioni devono usare tecniche constant-time e non devono far dipendere il flusso di esecuzione da dati segreti [6].

9.6 Scelta delle curve

Non tutte le curve ellittiche sono adatte alla crittografia. Una curva deve evitare strutture deboli, deve avere un sottogruppo di ordine sufficientemente grande e deve resistere agli attacchi noti. Per questo motivo, nei sistemi reali si usano curve standardizzate, i cui parametri sono stati pubblicati e analizzati [6].

La standardizzazione non elimina del tutto il problema della fiducia nei parametri. In alcuni casi, la comunità crittografica ha discusso la trasparenza con cui certe curve sono state generate. Per questa ragione, negli ultimi anni si è data maggiore attenzione a curve con parametri generati in modo verificabile e con implementazioni più semplici e meno soggette a errori [6].

9.7 La minaccia quantistica

Il limite più importante di ECC in prospettiva futura è rappresentato dai computer quantistici. I principali sistemi a chiave pubblica oggi usati, tra cui RSA, Diffie–Hellman ed ECC, si basano su problemi difficili per computer classici. Tuttavia, l'algoritmo di Shor mostra che un computer quantistico sufficientemente grande e fault-tolerant potrebbe risolvere in tempo polinomiale problemi considerati difficili per i computer classici [13]:

- la fattorizzazione di interi;
- il logaritmo discreto nei gruppi finiti;
- il logaritmo discreto su curve ellittiche.

Nel caso di ECC, questo significa che un avversario quantistico potrebbe recuperare d da

$$Q = dG.$$

Questo comprometterebbe direttamente la relazione tra chiave pubblica e chiave privata nei sistemi ECC [13]. I computer quantistici attuali non sono ancora in grado di eseguire attacchi di scala crittografica contro ECC, ma il rischio è rilevante per dati che devono rimanere segreti per molti anni.

9.8 Harvest Now, Decrypt Later

Un rischio collegato alla minaccia quantistica è chiamato *Harvest Now, Decrypt Later*. Un avversario può registrare oggi traffico cifrato che non riesce a decifrare immediatamente, conservarlo e tentare di decifrarlo in futuro quando saranno disponibili computer quantistici più potenti [13].

Questo scenario è particolarmente grave per dati con lunga durata di confidenzialità, come informazioni sanitarie, documenti governativi, segreti industriali o dati finanziari. Nel caso di protocolli basati su ECDH o ECDHE, un futuro avversario quantistico potrebbe recuperare i segreti effimeri dai punti pubblici scambiati e ricostruire la chiave di sessione [13].

9.9 Crittografia post-quantistica

La crittografia post-quantistica studia algoritmi eseguibili su computer classici ma progettati per resistere anche ad avversari dotati di computer quantistici. L'obiettivo è sostituire o affiancare RSA, Diffie–Hellman ed ECC con schemi basati su problemi matematici diversi, per i quali non siano noti algoritmi quantistici efficienti [13].

Le principali famiglie post-quantistiche si basano su diverse famiglie di problemi matematici [13]:

- problemi su reticoli;
- codici correttori di errore;
- funzioni hash;
- sistemi multivariati.

Negli ultimi anni, gli schemi basati su reticoli hanno avuto un ruolo particolarmente importante nella standardizzazione [13, 17, 18].

9.10 Standardizzazione e transizione

Il NIST ha avviato un processo di standardizzazione per selezionare algoritmi post-quantistici adatti all'uso pratico [13, 17, 18, 19]. Tra gli standard pubblicati vi sono:

- ML-KEM, derivato da Kyber, per incapsulamento di chiavi;
- ML-DSA, derivato da Dilithium, per firme digitali;

- SLH-DSA, derivato da SPHINCS+, per firme basate su hash.

La transizione alla crittografia post-quantistica non può essere immediata. Le infrastrutture esistenti sono profondamente basate su RSA ed ECC, e gli algoritmi post-quantistici hanno caratteristiche diverse, spesso con chiavi o firme più grandi.

Una strategia pratica è l'uso di sistemi ibridi, che combinano un algoritmo classico, come ECDHE, con un algoritmo post-quantistico, come ML-KEM [13, 17]. Il segreto finale può essere derivato da entrambe le componenti:

$$K = \text{KDF}(K_{\text{ECC}} \parallel K_{\text{PQC}}).$$

L'idea è che la comunicazione rimanga sicura se almeno una delle due componenti resta sicura. Durante il periodo di transizione, ECC continuerà probabilmente ad avere un ruolo importante, soprattutto nei sistemi in cui la minaccia quantistica non è immediata o in cui la compatibilità con infrastrutture esistenti è essenziale [13, 17].

9.11 Confronto finale

Il confronto tra RSA, ECC e crittografia post-quantistica può essere riassunto nella seguente tabella [12, 5, 13, 7]:

Famiglia	Problema base	Vantaggio	Limite principale
RSA	Fattorizzazione	Semplicità concettuale	Chiavi grandi, vulnerabile a Shor
ECC	ECDLP	Chiavi brevi, efficienza	Vulnerabile a Shor
PQC	Reticoli, hash, codici	Resistenza quantistica	Chiavi/firme spesso più grandi

Tabella 9.3: Confronto qualitativo tra RSA, ECC e crittografia post-quantistica.

ECC rappresenta quindi una soluzione molto efficace nel paradigma classico, ma non definitiva nel paradigma quantistico [5, 13]. La sua importanza rimane comunque notevole: ha reso più efficiente la crittografia a chiave pubblica e costituisce ancora oggi una componente centrale di molti sistemi reali.

Capitolo 10

Conclusioni

10.1 Risultati principali

Questa relazione ha analizzato la crittografia su curve ellittiche partendo dalle basi matematiche fino alle applicazioni moderne. Il punto centrale emerso è che ECC trasforma una struttura algebrica astratta in uno strumento pratico per la sicurezza digitale.

Dal punto di vista matematico, una curva ellittica in forma di Weierstrass è descritta da un'equazione del tipo

$$y^2 = x^3 + ax + b,$$

con una condizione di non singolarità che evita cuspidi e nodi. Sui numeri reali, questa equazione permette di comprendere geometricamente la somma tra punti tramite la regola della retta e della riflessione. Sui campi finiti, invece, la stessa struttura diventa utilizzabile in crittografia, perché le operazioni sono esatte, finite e implementabili in modo efficiente [5, 6].

L'operazione centrale è la moltiplicazione scalare

$$Q = dG.$$

Essa permette di generare chiavi pubbliche a partire da chiavi private, stabilire segreti condivisi tramite ECDH e produrre firme digitali tramite schemi come ECDSA. La sicurezza dipende dalla difficoltà di invertire questa operazione, cioè dal problema del logaritmo discreto su curve ellittiche [5, 10, 15, 6].

10.2 Rilevanza e limiti

La rilevanza pratica di ECC deriva soprattutto dalla sua efficienza. Rispetto a sistemi come RSA, ECC permette di ottenere livelli di sicurezza comparabili con chiavi più brevi, riducendo memoria, larghezza di banda e costo computazionale. Questo rende ECC adatta a protocolli di rete, dispositivi mobili, smart card, sistemi embedded e blockchain [7, 6].

Le applicazioni analizzate mostrano questa importanza. Nei protocolli di comunicazione sicura, ECC viene usata soprattutto tramite ECDHE per stabilire chiavi di

sessione con forward secrecy. In Bitcoin, invece, ECC è alla base della generazione delle chiavi, della derivazione degli indirizzi e della firma delle transazioni [14, 8].

Tuttavia, ECC non deve essere interpretata come una soluzione assoluta. La sua sicurezza è computazionale e dipende dalla difficoltà dell'ECDLP per i parametri scelti. Inoltre, errori pratici come cattiva generazione delle chiavi, riuso del nonce in ECDSA, mancata validazione dei punti o vulnerabilità side-channel possono compromettere un sistema anche se la matematica sottostante è solida [4, 15, 6].

Infine, ECC è vulnerabile alla minaccia quantistica. L'algoritmo di Shor mostra che un computer quantistico sufficientemente potente potrebbe risolvere in tempo polinomiale il logaritmo discreto su curve ellittiche. Per questo motivo, ECC dovrà convivere con schemi post-quantistici durante la fase di transizione verso nuove primitive crittografiche [13].

10.3 Conclusione finale

La crittografia su curve ellittiche rappresenta un esempio efficace di incontro tra matematica pura e ingegneria della sicurezza. Concetti come campi finiti, gruppi abeliani, punto all'infinito e logaritmo discreto diventano componenti operative di protocolli usati quotidianamente.

Il valore di ECC sta in questa combinazione: una struttura matematica ricca, un problema computazionale difficile e applicazioni pratiche efficienti. Comprendere ECC significa quindi comprendere uno dei meccanismi centrali della crittografia moderna, ma anche riconoscere che la sicurezza dipende sempre da assunzioni matematiche, parametri corretti, buone implementazioni e dall'evoluzione tecnologica.

Bibliografia

- [1] Luciano Margara. *Introduzione – Crittografia*. Materiale didattico del corso di Crittografia. 2022.
- [2] Luciano Margara. *Firma Digitale – Crittografia*. Materiale didattico del corso di Crittografia. 2022.
- [3] Luciano Margara. *Protocolli a chiave pubblica – Crittografia*. Materiale didattico del corso di Crittografia. 2026.
- [4] Luciano Margara. *Facile e Difficile – Crittografia*. Materiale didattico del corso di Crittografia. 2022.
- [5] Luciano Margara. *Crittografia su Curve Ellittiche – Crittografia*. Materiale didattico del corso di Crittografia. 2026.
- [6] Standards for Efficient Cryptography Group. *SEC 1: Elliptic Curve Cryptography, Version 2.0*. Standards for Efficient Cryptography. 2009.
- [7] Elaine Barker. *Recommendation for Key Management: Part 1 – General*. Special Publication 800-57 Part 1 Revision 5. National Institute of Standards e Technology, 2020. DOI: [10.6028/NIST.SP.800-57pt1r5](https://doi.org/10.6028/NIST.SP.800-57pt1r5).
- [8] Luciano Margara. *Bitcoin – Crittografia*. Materiale didattico del corso di Crittografia. 2026.
- [9] Standards for Efficient Cryptography Group. *SEC 2: Recommended Elliptic Curve Domain Parameters, Version 2.0*. Standards for Efficient Cryptography. 2010.
- [10] Luciano Margara. *Diffie Hellman – Crittografia*. Materiale didattico del corso di Crittografia. 2025.
- [11] Luciano Margara. *Protocollo a chiave pubblica: Elgamal – Crittografia*. Materiale didattico del corso di Crittografia. 2025.
- [12] Luciano Margara. *RSA – Crittografia*. Materiale didattico del corso di Crittografia. 2026.
- [13] Luciano Margara. *Crittografia Post-Quantistica – Crittografia*. Materiale didattico del corso di Crittografia. 2026.
- [14] Luciano Margara. *SSL/TLS – Crittografia*. Materiale didattico del corso di Crittografia. 2026.
- [15] National Institute of Standards and Technology. *Digital Signature Standard (DSS)*. Federal Information Processing Standards Publication FIPS 186-5. National Institute of Standards e Technology, 2023. DOI: [10.6028/NIST.FIPS.186-5](https://doi.org/10.6028/NIST.FIPS.186-5).

-
- [16] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. White paper. 2008.
 - [17] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Federal Information Processing Standards Publication FIPS 203. National Institute of Standards e Technology, 2024. DOI: [10.6028/NIST.FIPS.203](https://doi.org/10.6028/NIST.FIPS.203).
 - [18] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. Federal Information Processing Standards Publication FIPS 204. National Institute of Standards e Technology, 2024. DOI: [10.6028/NIST.FIPS.204](https://doi.org/10.6028/NIST.FIPS.204).
 - [19] National Institute of Standards and Technology. *Stateless Hash-Based Digital Signature Standard*. Federal Information Processing Standards Publication FIPS 205. National Institute of Standards e Technology, 2024. DOI: [10.6028/NIST.FIPS.205](https://doi.org/10.6028/NIST.FIPS.205).